

## **Экзамен СиФО ЭВМ**

- 1.Что относится к понятию архитектура команд?**
- 2.В чем состоит проблема семантического разрыва?**
- 3. Аккумуляторная архитектура системы команд, достоинства и недостатки.**
- 4. Стековая архитектура системы команд, достоинства и недостатки.**
- 5. Регистровая архитектура системы команд, достоинства и недостатки.**
- 6. Архитектура системы команд с выделенным доступом к памяти.**
- 7. Принципы CISC архитектуры**
- 8. Принципы RISC архитектуры.**
- 9. Типы команд.**
- 10.Способы адресации. Непосредственная и регистровая адресация.**
- 11.Способы адресации. Прямая и косвенная регистровая адресация.**
- 12.Способы адресации. Косвенная, автоинкрементная и автодекрементная адресация**
- 13.Способы адресации. Адресация со смещением (3 типа).**
- 14.Способы адресации. Страничная и блочная адресации.**
- 15.Способы адресации. Стековая адресация.**
- 16.Транзакция. Целевое назначение шин.**
- 17.Иерархия шин.**
- 18.Распределение линий шины управления.**
- 19.Алгоритмы смены приоритетов при арбитраже шин.**
- 20.Разновидности централизованного арбитража.**
- 21.Разновидности децентрализованного арбитража.**
- 22.Понятие протокола шины.**
- 23.Последовательность действий в процедуре квитирования установления связи.**
- 24.Синхронные и асинхронные шины, достоинства и недостатки.**
- 25.Методы повышения эффективности шин.**
- 26.Чем характеризуются ЗУ?**
- 27. Основная проблема памяти и способ её решения.**
- 28. Система ввода-вывода, понятие и задачи.**
- 29.Совмещенное адресное пространство СВВ, достоинства и недостатки.**
- 30.Конфликты в конвейере команд. Типы рисков.**
- 31.Риск по данным, типы конфликтов по данным.**
- 32.Предсказание переходов — статический и динамический.**
- 33.Стратегии статического перехода.**
- 34.Классификация архитектур системы команды (АСК)**
- 35.Принципы концепции ВМ фон Неймана. Какой из принципов можно назвать наиболее существенным?**
- 36.Принципы EPIC архитектуры (IA-64).**
- 37.Какие факторы влияют на длину команды? Возможные пути сокращения длины команды.**

38. В чём суть и достоинства конвейеризации транзакций?
39. Способы ускорения транзакций.
40. Методы доступа к ЗУ?
41. Иерархия запоминающих устройств.
42. Статические и динамические ОЗУ?
43. Блочная схема расслоения памяти.
44. Циклическая схема расслоения памяти.
45. Блочно-циклическая схема расслоения памяти.
46. Последовательный и конвейерный режим доступа к микросхеме ЗУ.
47. Регистровый и страничный режим доступа к микросхеме ЗУ.
48. Микросхемы SDRAM, отличия от асинхронных DRAM.
49. Энергонезависимые ОЗУ.
50. Специализированные ОЗУ.
51. Типы микросхем ПЗУ.
52. «Flash» ПЗУ.
53. Производительность иерархии памяти.
54. Ассоциативная память.
55. Кэш с полностью ассоциативным отображением.
56. Кэш с прямым отображением.
57. Кэш с множественно-ассоциативным отображением.
58. Эффективность множественно-ассоциативного отображения. Размеры строки.
- 59-60. Синхронизация данных. Метод сквозной/обратной записи.
61. Гарвардская архитектура организации. Преимущества.
62. Выделенное (изолированное) адресное пространство СВВ, достоинства и недостатки.
63. Структура ПУ. Модули ввода-вывода.
64. Структура и функции модуля ввода/вывода.
65. Программно управляемый ВВ.
66. Ввод/вывод по прерываниям.
67. Динамические схемы предсказания: А1.
68. Динамические схемы предсказания: А2.
69. Динамические схемы предсказания: А3.
70. Адрес команды условного перехода в качестве шаблона доступа к таблице предыстории переходов (РНТ).
71. Регистр глобальной истории в качестве шаблона доступа к таблице предыстории переходов (РНТ).
72. Регистр локальной истории в качестве шаблона доступа к таблице предыстории переходов (РНТ).
73. Комбинационный шаблон доступа к таблице предыстории переходов (РНТ).
74. Параллелизм. Уровни параллелизма.
75. Классификация Флинна.
76. Конвейеризация вычислений. Линейные и нелинейные конвейеры. Основные ступени конвейеров команд.

77. Программное и аппаратное устранение конфликтов по данным.
78. Риски по управлению. Потери производительности на переходах. Способы сокращения издержек на переходы.
79. Способы решения проблемы условного перехода
80. Переименование регистров.
81. Идентификация устройств при вводе/выводе по прерываниям.
82. Прямой доступ к памяти. Понятие и задачи КПП.
83. Переупорядочивание команд с использованием технологии «шелвинга»
84. Типы прерываний. Контроллеры прерываний ПК.
85. Типы регенерации динамических ЗУ.
86. Микросхемы DDR SDRAM.
87. Суперконвейерные процессоры.
88. Суперскалярные процессоры.
89. Операционные автоматы. Структура М- автомата и S- автоматов.
90. Операционные автоматы. Структура I-автомата.
91. Управляющие автоматы с жёсткой логикой. Автоматы Мили и Мура.
92. Управляющие автоматы с программируемой логикой.
93. Кодирование микрокоманд. Горизонтальный, вертикальный способы кодирования.
94. Кодирование микрокоманд. Смешанный способ кодирования команд.
95. Последовательность выполнения микропрограмм.
96. Организация памяти микропрограмм.
97. Закон Амдала.
98. Закон Густафсона.
99. Организация памяти вычислительных систем.
100. Модели архитектур распределённой памяти.
101. Когерентность -памяти.
102. Протокол MESI.
103. Организация виртуальной памяти.
104. Построение управляющих автоматов.
105. Способы кодирования микропрограмм.
106. Команды арифметической и логической обработки.
107. Команды управления потоком команд.
108. В чём состоит особенность SIMD-команд и в каком формате должны быть представлены операнды? Что такое «арифметика с насыщением» и где она применяется?
109. Особенности архитектуры ARM процессоров.
110. Организация виртуальной памяти. Многоуровневые таблицы трансляции адресов.
111. Архитектура ARM Cortex процессоров

## 1. Что относится к понятию архитектура команд?

Архитектура команд – интерфейс между всем выполняемым на машине ПО и аппаратным обеспечением.

Перечень видимых для программиста составных частей архитектуры системы команд:

- коды команд
- число и типы регистров
- форматы команд
- способы адресации и доступа к памяти
- условия возникновения исключений

АСК (ISA – Instruction Set Architecture) – очень важный уровень абстракции:

- интерфейс между аппаратурой и низко-уровневым ПО
- стандартизирует команды, шаблоны битов машинного языка и т.д.
- преимущества: позволяет реализовывать различные аппаратные варианты одной программной архитектуры
- недостатки: периодически препятствует внедрению новых технологий и прочих инноваций

Современные АСК: – 80x86/Pentium, PowerPC, MIPS, HP

Классификация архитектур системы команд (по месту хранения операндов):

Аккумуляторная Архитектура (EDSAC, 1950)

Регистровая Архитектура (IBM 360, 1964)

Архитектура с выделенным доступом к памяти Load/Store (CDC 6600, Cray I, 1963-76)

## 2. В чем состоит проблема семантического разрыва?

Современная технология программирования ориентирована на языки высокого уровня, главная цель которых — облегчить процесс программирования. Такой переход породил серьезную проблему: сложные операторы, характерные для ЯВУ, существенно отличаются от простых машинных операций, реализуемых в большинстве вычислительных машин. Следствием становится недостаточно эффективное выполнение программ на ВМ.

Факт различия принципов, лежащих в основе языков программирования высокого уровня и тех принципов, которые определяют архитектуру ЭВМ, называется *семантическим разрывом*.

Одной из дополнительных причин семантического разрыва является использование двоичной арифметики: современные вычислительные системы вынуждены затрачивать много времени на десятично-двоичные преобразования.

Таблица 2.1. Сравнительная оценка CISC-, RISC- и VLIW-архитектур

| Характеристика               | CISC                                               | RISC                                      | VLIW                                        |
|------------------------------|----------------------------------------------------|-------------------------------------------|---------------------------------------------|
| Длина команды                | Варируется                                         | Единая                                    | Единая                                      |
| Расположение полей в команде | Варируется                                         | Неизменное                                | Неизменное                                  |
| Количество регистров         | Несколько (часто специализированных)               | Много регистров общего назначения         | Много регистров общего назначения           |
| Доступ к памяти              | Может выполняться как часть команд различных типов | Выполняется только специальными командами | Выполняется только специальными командами - |

Таблица 2.1 позволяет оценить наиболее существенные различия в архитектурах типа CISC, RISC и VLIW.

## 3. Аккумуляторная архитектура системы команд, достоинства и недостатки.

Архитектура на базе аккумулятора исторически возникла одной из первых. В ней для хранения одного из операндов арифметической или логической операции в процессоре имеется выделенный регистр — аккумулятор. В этот же регистр заносится и

результат операции. Поскольку адрес одного из операндов предопределен, в командах обработки достаточно явно указать местоположение только второго операнда. Изначально оба операнда хранятся в основной памяти, и до выполнения операции один из них нужно загрузить в аккумулятор. После выполнения команды обработки результат находится в аккумуляторе и, если он не является операндом для последующей команды, его требуется сохранить в ячейке памяти.

Для загрузки в аккумулятор содержимого ячейки  $x$  предусмотрена команда загрузки  $\text{load } x$ . По этой команде информация считывается из ячейки памяти  $x$ , выход памяти подключается к входам аккумулятора и происходит занесение считанных данных в аккумулятор. Запись содержимого аккумулятора в ячейку  $x$  осуществляется командой сохранения  $\text{store } x$ , при выполнении которой выходы аккумулятора подключаются к шине, после чего информация с шины записывается в память.

Для выполнения операции в АЛУ производится считывание одного из операндов из памяти в регистр данных. Второй операнд находится в аккумуляторе. Выходы регистра данных и аккумулятора подключаются к соответствующим входам АЛУ. По окончании предписанной операции результат с выхода АЛУ заносится в аккумулятор.

Достоинствами аккумуляторной АСК можно считать короткие команды и простоту декодирования команд. Однако наличие всего одного регистра порождает многократные обращения к основной памяти.

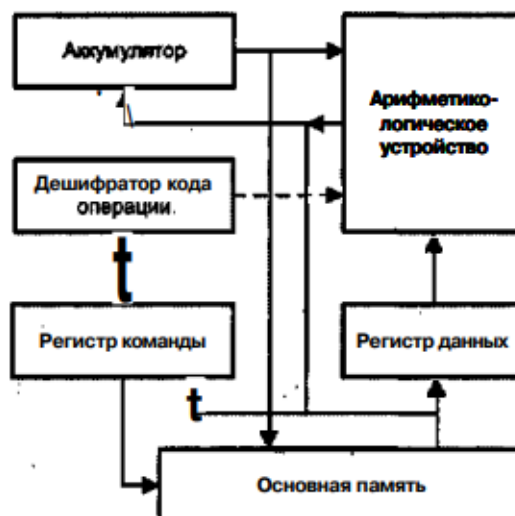


Рис. 2.7. Архитектура вычислительной машины на базе аккумулятора

#### 4. Стековая архитектура системы команд, достоинства и недостатки.

Стек – множество логически взаимосвязанных по принципу LIFO ячеек.

Две операции –  $\text{push}$  и  $\text{pop}$ . Сохранение из стека в память  $x$  по команде ( $\text{pop } x$ ).

Запись и чтение только в/из вершины стека.

Операнды помещаются в стек, в процессе операции они выталкиваются из стека.

Результат операции – в стек.

Для стековой архитектуры лучше всего подходит обратная польская нотация (Чарльз Хэмблин, 1957), разработанная на основе «польской нотации».

Информация в стек – из памяти или АЛУ. Результат – в АЛУ.

Достоинства:

- стек может быть неоднороден (верхние ячейки быстрые, нижние медленные).
- сокращение адр. части команд.
- компактный код
- простой компилятор
- простое декодирование команд

Недостатки:

- Компилятор не может создать эф. код (нет произвольного обращения к памяти).
- Стек – узкое место. Низкая производительность.

Последние годы – повышение интереса к стековой архитектуре, т.к. Java и Forth.  
ROSC (Removed Operand Set Computer) – новая реинкарнация этой архитектуры.



Рис. 2.6. Архитектура вычислительной машины на базе стека

## 5. Регистровая архитектура системы команд, достоинства и недостатки.

Массив регистров (РОН) – явно управляемый для хранения недавно использовавшихся данных. Размер регистров – обычно совпадает с машинным словом.

К любому регистру можно обратиться по номеру.

Меньшее количество регистров – меньше разрядов на кодирование номера регистра – короче команда. Операнды – либо в памяти, либо в регистрах.

Три подвида команд: Регистр-регистр, Регистр-память, Память-память.

Таблица 2.4. Сравнительная оценка вариантов размещения операндов

| Вариант                  | Достоинства                                                                                                                                                                              | Недостатки                                                                                                                                                                                                 |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Регистр-регистр<br>(0,3) | Простота реализации, фиксированная длина команд, простая модель формирования объектного кода при компиляции программ, возможность выполнения всех команд за одинаковое количество тактов | Большая длина объектного кода, из-за фиксированной длины команд часть разрядов в коротких командах не используется                                                                                         |
| Регистр-память<br>(1,2)  | Данные могут быть доступны без загрузки в регистры процессора, простота кодирования команд, объектный код получается достаточно компактным                                               | Потеря одного из операндов при записи результата, длинное поле адреса памяти в коде команды сокращает место под номер регистра, что ограничивает общее число РОН. CPI зависит от места размещения операнда |
| Память-память<br>(3,3)   | Компактность объектного кода, малая потребность в регистрах для хранения промежуточных данных                                                                                            | Разнообразие форматов команд и времени их исполнения, низкое быстродействие из-за обращения к памяти                                                                                                       |

Операции загрузки операндов в РОН – идентичны работе с аккумулятором. Отличие – выбор нужного регистра.

Выполнение операции АЛУ включает в себя:

- Выбор регистра первого операнда
- Определение местоположения второго операнда (память или регистр)
- Подача на вход АЛУ операндов и выполнение операции
- Выбор регистра результата и занесение в него результата операции из АЛУ.

Между АЛУ и массивом регистров должно быть минимум три шины.

Достоинства – компактность кода, высокая скорость.

Недостатки – более длинные инструкции

Регистровая АСК – преобладающая в настоящее время.

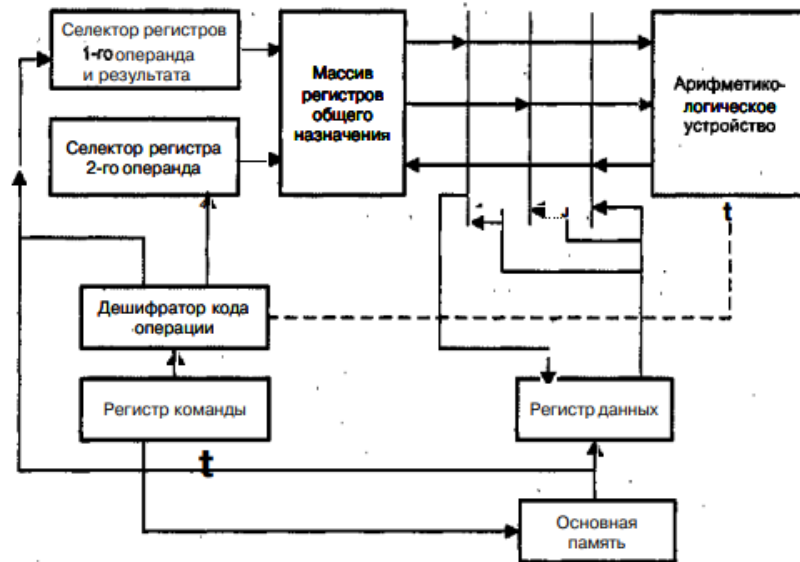


Рис. 2.8. Архитектура вычислительной машины на базе регистров общего назначения

## 6. Архитектура системы команд с выделенным доступом к памяти.

В архитектуре с выделенным доступом к памяти обращение к основной памяти возможно только с помощью двух специальных команд: load и store. Команда load (загрузка) обеспечивает считывание значения из основной памяти и занесение его в регистр процессора (в команде обычно указывается адрес ячейки памяти и номер регистра). Пересылка информации в противоположном направлении производится командой store (сохранение). Операнды во всех командах обработки информации могут находиться только в регистрах процессора (чаще всего в регистрах общего назначения). Результат операции также заносится в регистр. В архитектуре отсутствуют команды обработки, допускающие прямое обращение к основной памяти. Допускается наличие в АСК ограниченного числа команд, где операнд является частью кода команды.

Две из трех шин, расположенных между массивом РОН и АЛУ обеспечивают передачу в арифметико-логическое устройство операндов, хранящихся в двух регистрах общего назначения. Третья служит для занесения результата в выделенный для этого регистр. Эти же шины позволяют загрузить в регистры содержимое ячеек основной памяти и сохранить в ОП информацию, находящуюся в РОН.

АСК с выделенным доступом к памяти характерна для всех вычислительных машин с RISC-архитектурой. Команды в таких ВМ, как правило, имеют длину 32 бита и трехадресный формат. К достоинствам АСК следует отнести простоту декодирования и исполнения команды.

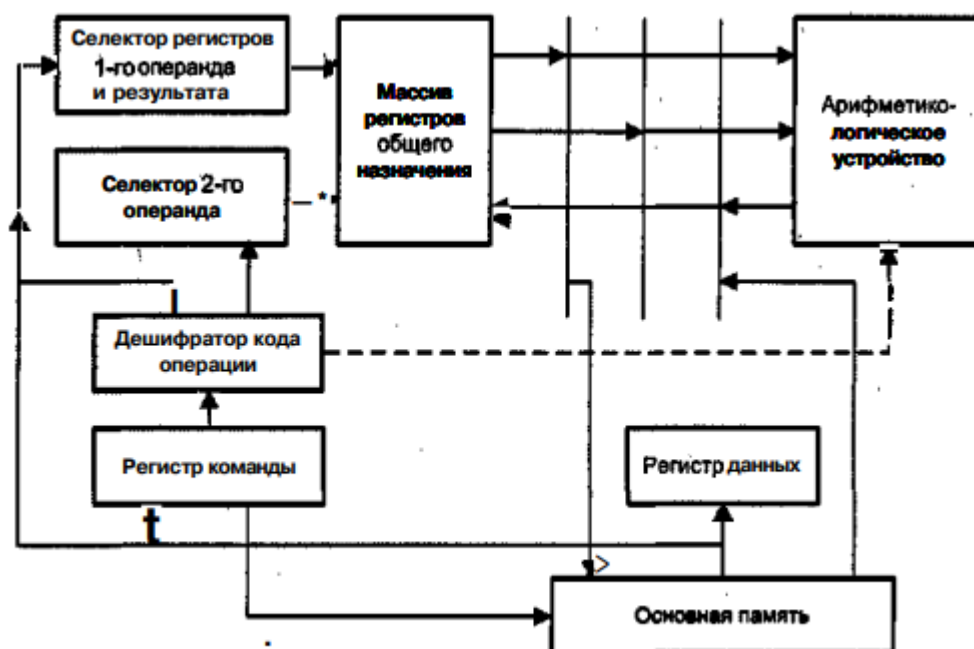


Рис. 2.9. Архитектура вычислительной машины с выделенным доступом к памяти

## 7. Принципы CISC архитектуры.

В вычислительных машинах типа CISC проблема семантического разрыва решается счет расширения системы команд, дополнения ее сложными командами, семантически аналогичными операторам ЯВУ. Основоположником CISC-архитектуры считается компания IBM.

Для CISC-архитектуры типичны:

- наличие в процессоре сравнительно небольшого числа регистров общего назначения;
- большое количество машинных команд, некоторые из них аппаратно реализуют сложные операторы ЯВУ;
- разнообразие способов адресации операндов;
- множество форматов команд различной разрядности;
- наличие команд, где обработка совмещается с обращением к памяти.

Недостатки CISC:

- высокая стоимость аппаратной части (особенно УУ)
- сложности с распараллеливанием вычислений

В CISC более сложные команды могут разбиваться на несколько частей, которые затем выполняются как последовательность микрокоманд. Эта операция снижает скорость работы, но она полезна для редко встречающихся команд.

## 8. Принципы RISC архитектуры.

RISC (Reduced Instruction Set Computer) - архитектура с сокращённым набором команд.

Детальный результатов упомянутых исследований привел к серьезному пересмотру традиционных решений, следствием чего стало появление RISC-архитектуры. Идея заключается в ограничении списка команд ВМ наиболее часто используемыми простейшими командами, оперирующими данными, размещенными только в регистрах процессорах. Обращение к памяти допускается лишь с помощью специальных команд чтения и записи. Резко уменьшено количество форматов команд и способов указания адресов операндов. Сокращение числа форматов команд и их простота, использование ограниченного количества способов адресации, отделение операций обработки данных от

операций обращения к памяти позволяет существенно упростить аппаратные средства ВМ и повысить их быстродействие. RISC-архитектура разрабатывалась таким образом, чтобы уменьшить время выч. за счет сокращения CPI и время пр. Как следствие, реализация сложных команд за счет последовательности из простых, но быстрых RISC-команд оказывается не менее эффективной, чем аппаратный вариант сложных команд в CISC-архитектуре.

Характерно:

- все команды выполняются непосредственно аппаратным обеспечением
- компьютер должен начинать выполнение большого количества команд (предполагается повышение производительности с помощью параллелизма)
- команды должны легко декодироваться (опять же для параллелизма, необходимо определять какие ресурсы нужны для выполнения команды)
- к памяти должны обращаться только команды загрузки и сохранения (чтобы не мешать параллелизму, ибо доступ к памяти - долгая история)
- должно быть большое количество регистров (ну чтобы к памяти слишком часто не обращаться)

## 9. Типы команд.

Команда – совокупность сведений, представленных в виде двоичных кодов, необходимых процессору для выполнения очередного шага.

*Команды пересылки данных.*

Примеры: передача данных между ЦП и ОП, внутри процессора, внутри памяти.

Содержит: - адреса источника и получателя операндов, размер данных, способ адресации каждого из операндов.

*Команды арифметической и логической обработки.*

Обеспечивают арифметическую и логическую обработку информации в различных формах ее представления.

*Команды SIMD — Single Instruction Multiple Data.*

Эти команды обрабатывают сразу две группы чисел. Операнды обычно представлены в упакованном формате.

*Команды работы со строками.*

Перемещение, сравнение и поиск строк.

*Команды преобразования.*

Изменение формата представления данных.

*Команды ввода/вывода.*

- Команды управления периферийными устройствами (ПУ). - Команды проверки состояния ПУ. - Команды ввода /вывода.

*Команды управления системой.*

Эти команды являются привилегированными командами, выполняются, когда ЦП находится в привилегированном состоянии либо выполняет привилегированную программу (обычно ОС). С помощью данных команд могут быть изменено состояние ряда регистров устройства управления.

*Команды управления потоком команд.*

Вместо инкрементации счётчика команд команда загружает в счётчик адрес следующей команды.

Три типа таких команд: - безусловные переходы; - условные переходы (ветвления); - вызовы процедур и возврат из процедур.

## 10.Способы адресации. Непосредственная и регистровая адресация.

При непосредственной адресации (НА) в адресном поле команды вместо адреса содержится непосредственно сам операнд. Этот способ применяется при выполнении арифметических операций, операций сравнения, для загрузки констант в регистры.

Когда операндом является число, оно обычно представляется в дополнительном коде. При записи в регистр, имеющий разрядность, превышающую длину непосредственного операнда, операнд размещается в младшей части регистра, а оставшиеся свободными позиции заполняются значением знакового бита операнда.

В адресном поле могут быть указаны только константы, доп. недостатком данного способа адресации – размер непосредственного операнда ограничен длиной адресного поля команды, которое в большинстве случаев меньше длины машинного слова.

Длина операнда редко превышает 16 бит. Т.о, в подавляющем числе случаев 16 разрядов вполне достаточно, хотя для вычисления адресов const может быть длиннее.

Основная проблема – не каждый операнд можно передать непосредственно – адр. часть команды как правило меньше машинного слова. 50-60% - до 8 бит, 70-80% - до 16 бит.

Плюсы – малое время выполнения команды, экономия памяти.

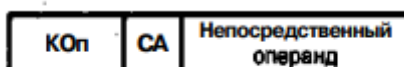


Рис. 2.45. Непосредственная адресация

Регистровая адресация (РА) напоминает прямую адресацию. Адресное поле инструкции указывает на регистр процессора. Обычно размер адресного поля в данном случае составляет три или четыре бита, что позволяет указать соответственно на один из 8 или 16 регистров общего назначения (РОН). Двумя основными преимуществами регистровой адресации являются: короткое адресное поле в команде и исключение обращений к памяти.

Плюсы – размер адресного поля 3-4 бита для 8-16 РОН. Исключение обращений к памяти.

Минусы – мало РОН.

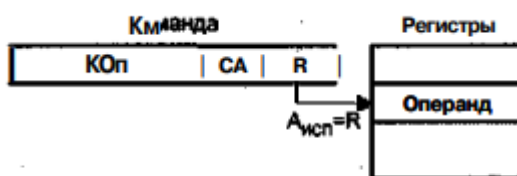


Рис. 2.50. Регистровая адресация

Способ адресации – это способ формирования исполнительного адреса по адресному коду команды.

## 11.Способы адресации. Прямая и косвенная регистровая адресация.

При прямой (абсолютной) адресации (ПА) адресный код указывает номер ячейки памяти, к которой производится обращение, то есть адресный код = исполнит. адрес.

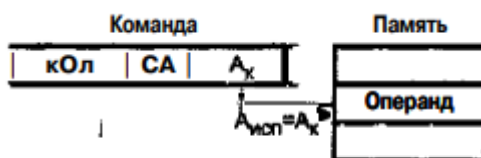


Рис. 2.48. Прямая адресация

При всей простоте использования способ имеет существенный недостаток - ограниченный размер адресного пространства, так как для адресации к памяти большой емкости нужно «длинное» адресное поле. Однако более существенным несовершенством можно считать то, что адрес, указанный в команде, не может быть изменен в процессе вычислений (во всяком случае, такое изменение не рекомендуется). Это ограничивает возможности по произвольному размещению программы в памяти.

Плюсы – простота.

Существенный недостаток – ограниченный размер адресного пространства. Адрес в команде не может быть изменён в процессе вычислений -> ограничение на размещение программы в ОЗУ.

Косвенная регистровая адресация (КРА) представляет собой косвенную адресацию, где исполнительный адрес операнда хранится не в ячейке основной памяти, а в регистре процессора. Соответственно, адресное поле команды указывает не на ячейку памяти, а на регистр.

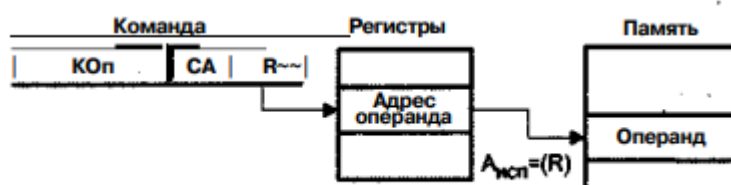


Рис. 2.51. Косвенная регистровая адресация

Достоинства и ограничения косвенной регистровой адресации те же, что и у обычной косвенной адресации (косвенный адрес в процессе выполнения программы можно изменять, упрощение обработки массивов и списков, а также передачи параметров подпрограмм), но благодаря тому, что косвенный адрес хранится не в памяти, а в регистре, для доступа к операнду требуется на одно обращение к памяти меньше.

Плюсы аналогичны обычной косвенной адресации – можно изменять адреса операндов в процессе вычислений. На одно обращение к памяти меньше.

**Способ адресации** – это способ формирования исполнительного адреса по адресу коду команды.

## 12. Способы адресации. Косвенная, автоинкрементная и автодекрементная адресация

Одним из путей преодоления проблем, свойственных прямой адресации, может служить прием, когда с помощью ограниченного адресного поля команды указывается адрес ячейки, в свою очередь, содержащей полноразрядный адрес операнда. Этот способ известен как косвенная адресация (КА).

При косвенной адресации содержимое адресного поля команды остается неизменным в то время как косвенный адрес в процессе выполнения программы можно изменять. Это позволяет проводить вычисления, когда адреса операндов заранее неизвестны и появляются лишь в процессе решения задачи. Дополнительно такой прием упрощает обработку массивов и списков, а также передачу параметров подпрограмм.



Рис. 2.49. Косвенная адресация

Плюсы – можно изменять адреса операндов в процессе вычислений.

Недостатком косвенной адресации является необходимость в двукратном обращении к памяти: сначала для извлечения адреса операнда, а затем для обращения к операнду. + задействуется лишняя ячейка памяти для хранения исполнительного адреса операнда.

Автоиндексирование с увеличением содержимого индексного регистра носит название автоинкрементной адресации. Автоиндексирование с уменьшением содержимого индексного регистра носит название автодекрементной адресации.

В большинстве ВМ увеличение или уменьшение содержимого индексного регистра до или после обращения к нему осуществляется автоматически как часть машинного цикла. Такой прием называется автоиндексированием.

При индексной адресации (ИА) подполе содержит адрес ячейки памяти, а регистр (указанный явно или неявно) — смещение относительно этого адреса. Как видно, этот способ адресации похож на базовую регистровую адресацию. Поскольку при индексной адресации в поле полноразрядный адрес ячейки памяти, играющий роль базы, длина этого поля больше, чем при базовой регистровой адресации.

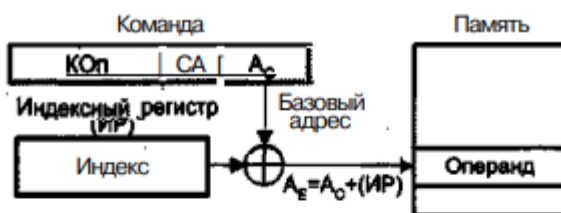


Рис. 2.87. Индексная адресация с индексным регистром

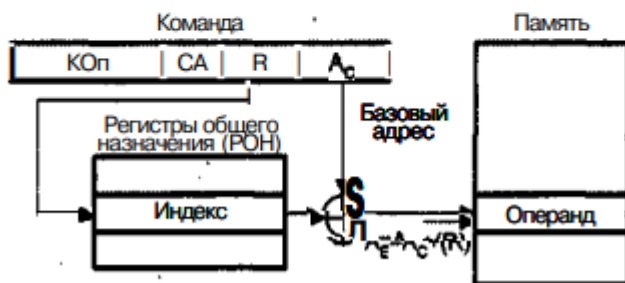


Рис. 2.58. Индексная адресация с использованием одного из РОП

Способ адресации – это способ формирования исполнительного адреса по адресному коду команды.

### 13.Способы адресации. Адресация со смещением (3 типа).

В адресации со смещением исполнительный адрес формируется в результате суммирования содержимого адресного поля содержимым одного или нескольких регистров процессора. Адресация со смещением предполагает, что адресная часть команды включает в себя как минимум одно поле. В нем содержится константа, смысл которой в разных вариантах может меняться. Константа может представлять собой некий базовый адрес, к которому добавляется хранящееся в регистре смещение.

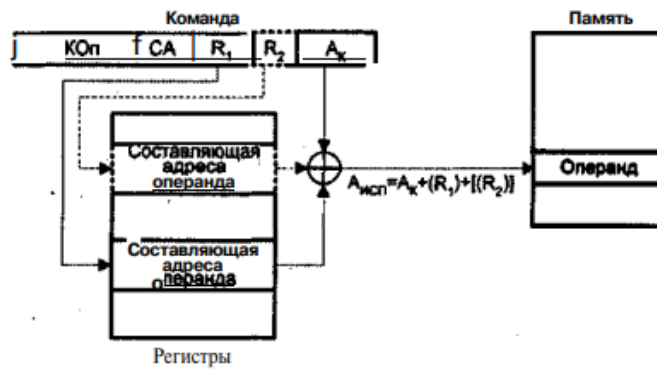


Рис. 2.52. Адресация со смещением

При относительной адресации (ОА) для получения исполнительного адреса операнда содержимое подполя  $A_x$  складывается с содержимым счетчика команд. Т.о, адресный код в команде представляет собой смещение относительно адреса текущей команды. Следует отметить, что в момент вычисления исполнительного адреса операнда в счетчике команд может уже быть сформирован адрес следующей команды, что нужно учитывать при выборе величины смещения. Обычно подполе как двоичное число в дополнительном коде.

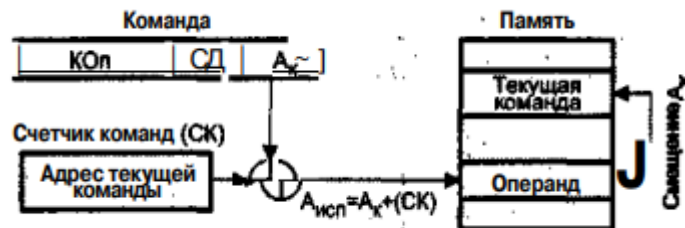


Рис. 2.53. Относительная адресация

В случае базовой регистровой адресации (БРА) регистр, называемый базовым, содержит полноразрядный адрес, а подполе  $A_c$  смещение относительно адреса. Ссылка на базовый регистр может быть явной или неявной. В некоторых ВМ имеется специальный базовый регистр и его использование является неявным. Базовую регистровую адресацию обычно используют для доступа к элементам массива, положение которого в памяти в процессе вычислений может меняться.

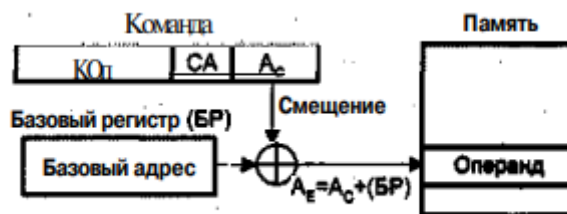


Рис. 2.54. Базовая регистровая адресация с базовым регистром

При индексной адресации (ИА) подполе содержит адрес ячейки памяти, а регистр (указанный явно или неявно) — смещение относительно этого адреса. Как видно, этот способ адресации похож на базовую регистровую адресацию. Поскольку при индексной адресации в поле полноразрядный адрес ячейки памяти, играющий роль базы, длина этого поля больше, чем при базовой регистровой адресации.

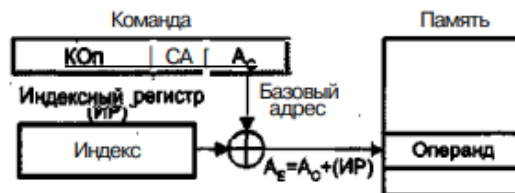


Рис. 2.87. Индексная адресация с индексным регистром

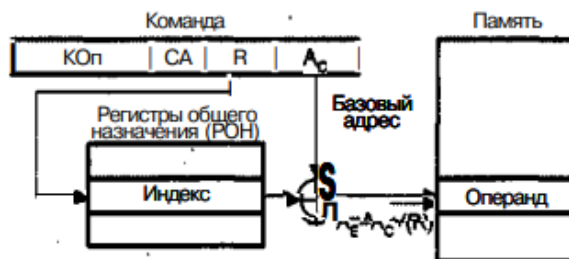


Рис. 2.58. Индексная адресация с использованием одного из Rn

Способ адресации – это способ формирования исполнительного адреса по адресному коду команды.

## 14. Способы адресации. Страничная и блочная адресации.

Страничная адресация (СТА) предполагает разбиение адресного пространства на страницы. Страница определяется своим начальным адресом, выступающим в качестве базы. Старшая часть этого адреса хранится в специальном регистре — регистре адреса страницы (РАС). В адресном коде команды указывается смещение внутри страницы, рассматриваемое как младшая часть исполнительного адреса.

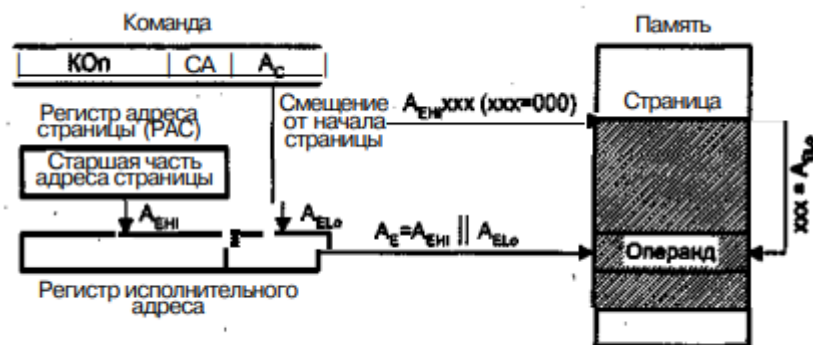


Рис. 2.59. Страничная адресация

Блочная адресация используется в командах, для которых единицей обработки служит блок данных, расположенных в последовательных ячейках памяти. Этот способ очень удобен при работе с внешними запоминающими устройствами и в операциях с векторами. Для описания блока обычно берется адрес ячейки, где хранится первый или последний элемент блока, и общее количество элементов блока, заданное числом байтов или ячеек. Вместо длины блока может использоваться специальный признак «конец блока», помещаемый за последним элементом блока. Удобен для работы с внешними ЗУ, операциях с векторами.

Способ адресации – это способ формирования исполнительного адреса по адресному коду команды.

## 15. Способы адресации. Стековая адресация

Стек представляет собой группу последовательно пронумерованных регистров (*аппаратурный стек*) или ячеек памяти, снабженных указателем стека (обычно регистром) (УС), в котором автоматически при записи и считывании устанавливается номер (адрес) последней занятой ячейки стека (вершины стека). При операции записи

заносимое в стек слово помещается в следующую по порядку свободную ячейку стека, а при считывании из стека извлекается последнее поступившее в него слово. Таким образом, в стеке реализуется правило «*последний пришел - первый ушел*».

Указанное правило при обращении к стеку реализуется автоматически, и поэтому при операциях со стеком возможно безадресное задание операнда — команда не содержит адреса ячейки стека, но содержит адрес (или он подразумевается) ячейки памяти или регистра, откуда слово передается в стек или куда помещается из стека.

При выполнении команды передачи в стек слова из регистра или ячейки ОП сначала указатель стека увеличивается на 1 (в *перевернутом стеке* уменьшается на 1), а затем слово помещается в ячейку стека, указываемую УС. При команде загрузки из стека регистра или ячейки памяти сначала слово извлекается из вершины стека, а затем указатель стека уменьшается на 1 (в перевернутом стеке увеличивается на 1). Как это ни кажется на первый взгляд удивительным, но при соответствующем расположении операндов в стеке можно вычислять выражения полностью безадресными командами, указывающими только вид операции. Такая команда извлекает из стека в соответствии с кодом операции один или два операнда, выполняет над ними предписанную операцию и заносит результат в стек.

**Способ адресации – это способ формирования исполнительного адреса по адресному коду команды.**

Используется в ВМ со стековой архитектурой.

**Стековый доступ к памяти происходит в большинстве процессоров в следующих ситуациях:**

1) При выполнении команд стекового доступа: "поместить в стек" push или "извлечь из стека" pop.

2) При выполнении команды вызова подпрограммы и при возврате из нее.

3) При входе в прерывание и возврате из него.

Положение стека в адресуемой памяти определяется содержимым указателя стека.

Содержимое указателя стека используется как адрес операнда-приемника при записи или как адрес операнда-источника при считывании из стека.

При обращении к стеку автоматически модифицируется и содержимое указателя стека, чтобы обеспечить следующее обращение к очередной ячейке стека.

Обращение к стеку - косвенно-регистровая адресация через указатель стека с автоиндексацией.

## **16. Транзакция. Целевое назначение шин.**

Взаимосвязь частей ВМ и ее «общение» с внешним миром обеспечиваются системой шин. Большинство машин содержат несколько различных шин, каждая из которых оптимизирована под определенный вид коммуникаций. Часть шин скрыта внутри интегральных микросхем или доступна только в пределах печатной платы. Некоторые шины имеют доступные извне точки, с тем чтобы к ним легко можно было подключить дополнительные устройства, причем большинство таких шин не просто доступны, но и отвечают определенным стандартам, что позволяет подсоединять к шине устройства различных производителей.

Шину образует набор коммуникационных линий, каждая из которых способна передавать сигналы, представляющие двоичные цифры 1 и 0. По линии может пересылаться развернутая во времени последовательность таких сигналов. При совместном использовании несколько линий могут обеспечить одновременную (параллельную) передачу двоичных чисел.

Этим стремятся подчеркнуть, что шина ведет себя как магистраль, способная обеспечить всевозможные виды трафика

Операции на шине называют транзакциями. Основные виды транзакций - транзакции чтения и транзакции записи. Если в обмене участвует устройство

ввода/вывода, можно говорить о транзакциях ввода и вывода, по сути эквивалентных транзакциям чтения и записи соответственно. Шинная транзакция включает в себя две части: посылку адреса и прием (или посылку) данных.

Целевое назначение шин:

- Шины «процессор-память»
- Шины ввода/вывода
- Системные шины

Шина «процессор-память»

Связь между ЦП и ОП (основной памятью)- Front-Side Bus (FSB).

Шина для связи процессора с -памятью второго уровня – Back Side Bus (BSB).

Максимизация пропускной способности и минимизация длины шины.

Шины ввода/вывода

Соединение между ЦП и устройствами В/В-да.

Многообразие устройств -> стандартизация и унификация шин.

За исключением видеосистем – шины в/в не особо быстры.

Минимизация конструктива шины (разъёмов и т.п.). Меньше линий, но больше длина (SCSI).

Системная шина

Физ. и лог. объединение всех устр-в ВМ (backplane bus). Обычно несколько сот линий.

## 17. Иерархия шин.

Если к шине подключено большое число устройств, ее пропускная способность падает, поскольку слишком частая передача прав управления шиной от одного устройства к другому приводит к ощутимым задержкам. => во многих ВМ предпочтение отдается использованию нескольких шин, образующих определенную иерархию.

### Вычислительная машина с одной шиной

В структурах взаимосвязей с одной шиной имеется одна системная шина, обеспечивающая обмен информацией между процессором и памятью, а также между УВВ, с одной стороны, и процессором либо памятью - с другой (рис. 4.5).



Рис. 4.5. Структура взаимосвязей с одной шиной

Для такого подхода характерны простота и низкая стоимость. Однако одношинная организация не в состоянии обеспечить высокие интенсивность и скорость транзакций, причем «узким местом» становится именно шина.

## Вычислительная машина с двумя видами шин

Хотя контроллеры устройств ввода/вывода (УВВ) могут быть подсоединены непосредственно к системной шине, больший эффект достигается применением одной или нескольких шин ввода/вывода (рис. 4.6). УВВ подключаются к шинам ввода/вывода, которые берут на себя основной трафик, не связанный с выходом на процессор или память. *Адаптеры шин* обеспечивают буферизацию данных при их пересылке между системной шиной и контроллерами УВВ. Это позволяет ВМ поддерживать работу множества устройств ввода/вывода и одновременно «развязать» обмен информацией по тракту процессор-память и обмен информацией с УВВ.



Рис. 4.6. Структура взаимосвязей с двумя видами шин

## Вычислительная машина с тремя видами шин

Для подключения быстродействующих периферийных устройств в систему шин может быть добавлена высокоскоростная шина расширения (рис. 4.7).

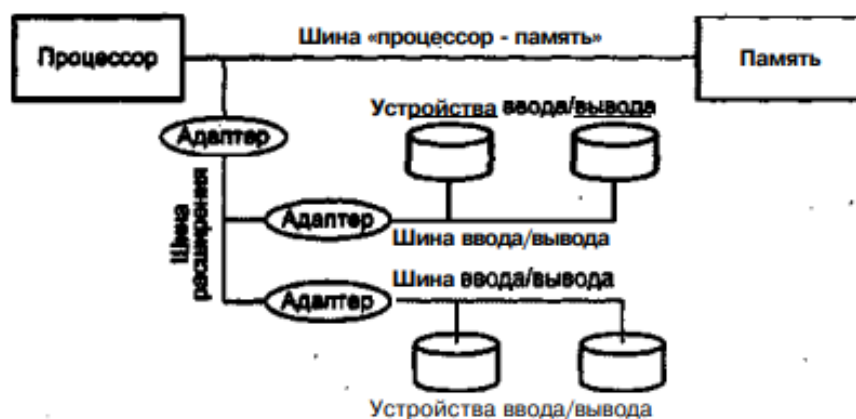
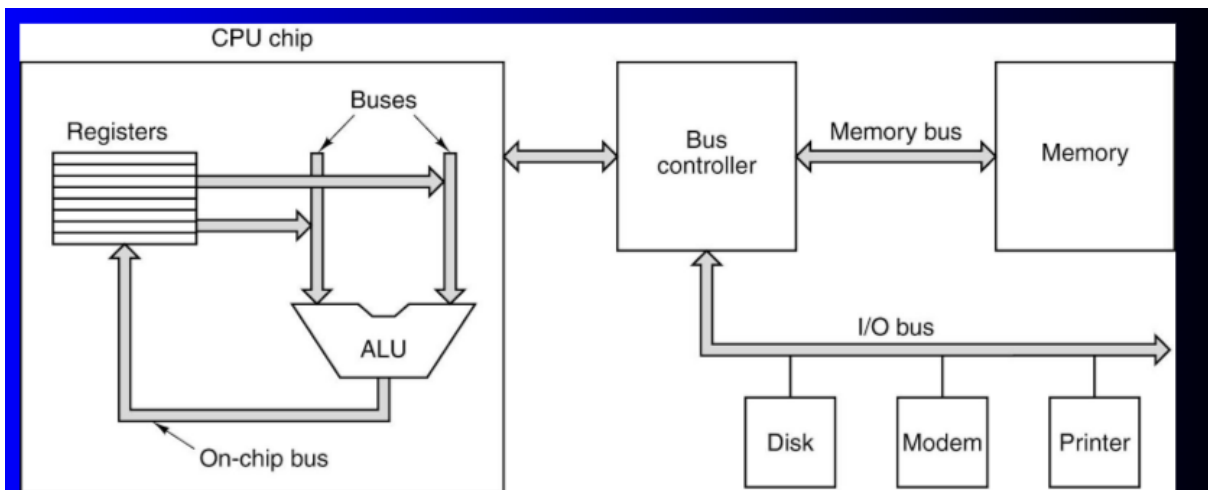


Рис. 4.7. Структура взаимосвязей с тремя видами шин

Шины ввода/вывода подключаются к шине расширения, а уже с нее через адаптер к шине «процессор-память». Схема еще более снижает нагрузку на шину «процессор-память». Такую организацию шин называют *архитектурой с «пристройкой»* (mezzanine architecture).



## A computer system with multiple buses.

| Master      | Slave       | Example                                    |
|-------------|-------------|--------------------------------------------|
| CPU         | Memory      | Fetching instructions and data             |
| CPU         | I/O device  | Initiating data transfer                   |
| CPU         | Coprocessor | CPU handing instruction off to coprocessor |
| I/O         | Memory      | DMA (Direct Memory Access)                 |
| Coprocessor | CPU         | Coprocessor fetching operands from CPU     |

## Examples of bus masters and slaves.

### 18. Распределение линий шины управления.

#### Шина управления (ШУ):

*Линии сигналов управления транзакциями*—

- тип выполняемой транзакции (чт. или запись),
- количество байтов, передаваемых по ШД, если часть слова – то какие байты.
- какой тип адреса на ША
- какой протокол передачи должен использоваться

Всего на данную группу - от 2 до 8 линий.

*Линии информации состояния (статуса)* –

от одной до четырёх линий для передачи ведомым информации ведущему.

*Линии арбитража* – от 3-х до 11 линий.

*Линии прерывания* – запросы от ведомых к ведущему на обслуживание (обычно одна две-линии) + доп. Арбитраж (если не используются линии пред. группы).

*Линии для организации последовательных локальных сетей* – обычно от 1 до 4 линий (последовательная передача значительно медленнее параллельной – выгоднее добавить пару линий чем загружать основные), иногда могут заменить ША и ШД, служить для реализации спец. функций – обработка прерываний или сортировка приоритетов задач.

*Линии позиционного кода* – (от 4 до 5 линий) – для передачи уникального позиционного кода дочерних плат.

*Линии тактирования и синхронизации* – от 2 до 6 линий – в зависимости от протокола (асинхронный, синхронный).

*Линии питания и заземления* – от 2 до 20.

## **19. Алгоритмы смены приоритетов при арбитраже шин.**

Несколько ведущих на одной шине -> конфликт интересов. Значит, необходимо присвоение приоритетов.

**Статический приоритет** - устройства с высшим приоритетом могут полностью блокировать шину.

**Динамический** - шанс есть у каждого.

Наибольшее распространение получили следующие алгоритмы динамического изменения приоритетов:

- простая циклическая смена приоритетов;
- циклическая смена приоритетов с учетом последнего запроса;
- смена приоритетов по случайному закону;
- схема равных приоритетов;
- алгоритм наиболее давнего использования;
- алгоритм очереди (первым пришел — первым обслужен);
- алгоритм фиксированного кванта времени.

В алгоритме простой циклической смены приоритетов после каждого цикла арбитража все приоритеты понижаются на один уровень, при этом устройство, имевшее ранее низший уровень приоритета, получает наивысший приоритет.

В схеме циклической смены приоритетов с учетом последнего запроса все возможные запросы упорядочиваются в виде циклического списка. После обработки очередного запроса обслуженному ведущему назначается низший уровень приоритета.

Следующее в списке устройство получает наивысший приоритет, а остальным устройствам приоритеты назначаются в убывающем порядке, согласно их следованию в циклическом списке.

В обеих схемах циклической смены приоритетов каждому ведущему обеспечивается шанс получить шину в свое распоряжение, однако большее распространение получил второй алгоритм.

При смене приоритетов по случайному закону после очередного цикла арбитража с помощью генератора псевдослучайных чисел каждому ведущему присваивается случайное значение уровня приоритета.

В схеме равных приоритетов при поступлении к арбитру нескольких запросов каждый из них имеет равные шансы на обслуживание. Возможный конфликт разрешается арбитром. Такая схема принята в асинхронных системах.

В алгоритме наиболее давнего использования (LRU, Least Recently Used) после каждого цикла арбитража наивысший приоритет присваивается ведущему, который дольше чем другие не использовал шину.

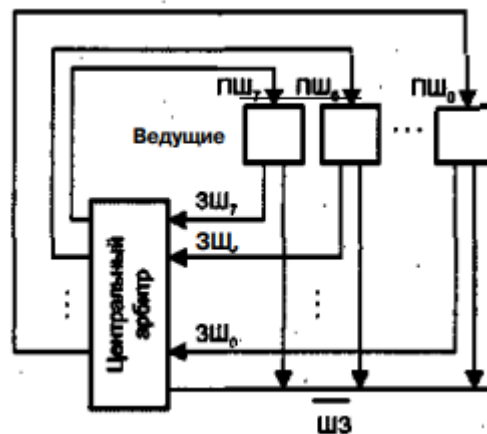
В алгоритме очереди запросы обслуживаются в порядке очереди, образовавшейся к моменту начала цикла арбитража. Сначала обслуживается первый запрос в очереди, то есть запрос, поступивший раньше остальных. Аппаратурная реализация алгоритма связана с определенными сложностями, поэтому используется он редко.

В алгоритме фиксированного кванта времени каждому ведущему для захвата шины в течение цикла арбитража выделяется определенный квант времени. Если ведущий в этот момент не нуждается в шине, выделенный ему квант остается не использованным. Такой метод наиболее подходит для шин с синхронным протоколом.

## 20. Разновидности централизованного арбитража

При централизованном арбитраже в системе имеется специальное устройство — центральный арбитр, - ответственное за предоставление доступа к шине только одному из запросивших ведущих. Это устройство, называемое иногда центральным контроллером шины, может быть самостоятельным модулем или частью ЦП. Наличие на шине только одного арбитра означает, что в централизованной схеме имеется единственная точка отказа. В зависимости от того, каким образом ведущие устройства подключены к центральному арбитру возможные схемы централизованного арбитража можно подразделить на параллельные и последовательные.

В параллельном варианте центральный арбитр связан с каждым потенциальным ведущим индивидуальными двухпроводными трактами. Поскольку запросы к центральному арбитру могут поступать независимо и параллельно, данный вид арбитража называют централизованным параллельным арбитражем или централизованным арбитражем независимых запросов.



Второй вид централизованного арбитража известен как централизованный последовательный арбитраж. В последовательных схемах для выделения запроса с наивысшим приоритетом используется один из сигналов, поочередно проходящий через цепочку ведущих, чем и объясняется другое название - цепочечный или гирляндный арбитраж.

В зависимости от того, какой из сигналов используется для целей арбитража, различают три основных типа схем цепочечного арбитража: с цепочкой для сигнала предоставления шины, с цепочкой для сигнала запроса шины (ЗШ) и с цепочкой для дополнительного сигнала разрешения (ПШ). Наиболее распространена схема цепочки для сигнала ПШ.

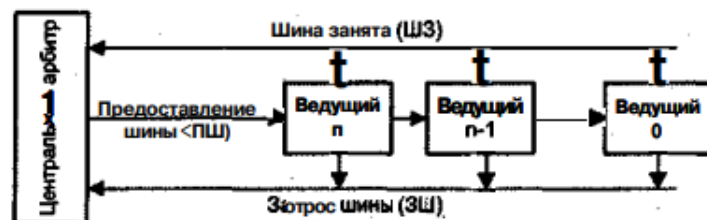


Рис. 4.12. Централизованный последовательный арбитраж с цепочкой для сигнала предоставления шины

Еще раз отметим, что очередной ведущий не может приступить к управлению шиной до момента ее освобождения. Центральный арбитр не должен формировать сигнал ПШ вплоть до этого момента.

Цепочечная реализация предполагает статическое распределение приоритетов. Наивысший приоритет имеет ближайшее к арбитру ведущее устройство (устройство, на которое арбитр дал сигнал ПШ). Далее приоритеты ведущих в цепочке последовательно понижаются.

Основное достоинство цепочечного арбитража заключается в простоте реализации и в малом количестве используемых линий. Последовательные схемы арбитража позволяют легко наращивать число устройств, подключаемых к шине.

Схеме тем не менее присущи существенные недостатки. Прежде всего, последовательное прохождение сигнала по цепочке замедляет арбитраж, причем время арбитража растет пропорционально длине цепочки. Статическое распределение приоритетов может привести к полному блокированию устройств с низким уровнем приоритета (расположенных в конце цепочки). Наконец, как и параллельный вариант, централизованный последовательный арбитраж не очень удобен в плане диагностики работы шины

## 21. Разновидности децентрализованного арбитража.

Кольцевая схема с циклической сменой приоритетов.

Переход к следующему ведущему – со сменой приоритетов. Текущий ведущий в след. цикле – наименьший приоритет, его сосед справа – наивысший, остальные – на 1 меньше, чем у соседа слева.

Циклическая смена с учётом последнего запроса.



Слайд 36

Текущий ведущий генерирует ПШ, который проходит через все схемы ЛА. Если имеется ещё один ведущий, который просит шину (ЗШ), то его ЛА не пропускает ПШ к другим устройствам. Текущий ведущий, «потеряв» ПШ на своём входе, должен при первой возможности освободить шину. Запрашивающий – её займет.

Чаще встречается иная схема децентр-го арбитража. Арбитражные линии и устройства по схеме «монтажное ИЛИ». Каждому ведущему – уникальный номер (уровень приоритета). Запрашивающие шину выставляют на арб. линии свой номер. Каждый из запрашивающих, обнаружив на линиях номер с более высоким приоритетом снимает младшие биты своего номера. В итоге (может понадобится несколько итераций) на линии остаётся только номер с наиболее высоким приоритетом. Ведущий, распознавший на линиях свой номер – захватывает шину. Способ называется – *распределённый арбитраж с самостоятельным выбором*. Модуль равнодоступности – последний ведущий не может участвовать в след. цикле арбитража. Шины – FutureBus, NuBus, MMultiBus, FastBus.

## 22. Понятие протокола шины

### Протокол шины

Сигналы на шине – перекося – прежде чем реагировать, все ведомые должны знать с какого момента поступивший адрес можно считать достоверным.

С данными ещё сложнее – могут передаваться в двух направлениях. При чтении – доп. задержка на выдачу ведомым необходимых ведущему данных на шину.

Метод информирования о достоверности адреса, данных, управляющей информации и информации состояния – протокол шины.

Два класса протоколов – **синхронный** (все сигналы привязаны к импульсам ГТИ) и **асинхронный** – для каждой группы сигналов свой сигнал подтверждения достоверности.

# Протокол шины

**Синхронные шины:** (доп. неудобства) ведущий не знает, что ответил ведомый, ведущий должен работать со скоростью самого медленного из устр-в, участвующих в пересылке данных.

**Асинхронные шины:** позволяют избегать указанных выше недостатков.

Начало очередного события определяется не ТИ, а окончанием предыдущего. Каждая совокупность сигналов, помещаемых на шину, сопровождается синхронизирующим сигналом (стробом).

Синхросигналы от ведомого – *квитирующие сигналы* (handshakes) или *подтверждения сообщений* (acknowledges).

## 23. Последовательность действий в процедуре квитирования установления связи.

### Квитирование установления связи

Начальный обмен служебной информацией между двумя системами перед началом и во время передачи данных, выполняемый для обеспечения правильности передачи данных.

Метод квитирования установления связи (например, XON/XOF) является частью полного протокола передачи данных.

Последовательный (асинхронный) протокол передачи данных может включать в себя метод квитирования установления связи, установку скорости передачи, контроля четности, количества битов данных и числа стоповых битов.

### Концепция квитирования

В рамках соединения правильность передачи каждого сегмента должна подтверждаться квитанцией получателя. *Квитирование* - это один из традиционных методов обеспечения надежной связи. Идея квитирования состоит в следующем.

Для того, чтобы можно было организовать повторную передачу искаженных данных отправитель нумерует отправляемые единицы передаваемых данных (далее для простоты называемые кадрами). Для каждого кадра отправитель ожидает от приемника так называемую положительную квитанцию - служебное сообщение, извещающее о том, что исходный кадр был получен и данные в нем оказались корректными. Время этого ожидания ограничено - при отправке каждого кадра передатчик запускает таймер, и если по его истечению положительная квитанция не получена, то кадр считается утерянным. В некоторых протоколах приемник, в случае получения кадра с искаженными данными должен отправить отрицательную квитанцию - явное указание того, что данный кадр нужно передать повторно.

Существуют два подхода к организации процесса обмена положительными и отрицательными квитанциями: с простоями и с организацией "окна".

[http://citforum.ru/nets/ip/glava\\_6.shtml](http://citforum.ru/nets/ip/glava_6.shtml)

ссылка - сайт со второго скрина(описываются два подхода, если посчитаешь, что не хватит).

Квитирование широко применяется в параллельных интерфейсах (в том же LPT-порте, шинах расширения), для чего используются специальные интерфейсные линии.

## 24. Синхронные и асинхронные шины, достоинства и недостатки.

### Синхронные шины

**Синхронные шины:** следует добиваться, чтобы ТИ доставлялись к каждому разъёму шины практически одновременно.

ТИ должен выбираться так, чтобы любой сигнал на шине мог быть доставлен в любую её точку до завершения тактового периода -> чем короче шина, тем м.б. выше частота ТИ.

**Синхронн**ые протоколы – проще, требуют меньше сигн. линий, но менее гибки – привязаны к конкретной тактовой частоте (уровню технологий). Из-за проблемы перекоса синхросигналов **синхронн**ые шины не могут быть длинными.

Обычно синхр. шины – «процессор-память».

# Асинхронные шины

Скорость асинхронной пересылки определяется ведомым -> шины самосинхронизирующиеся, поэтому успешно могут применяться и новые и старые устройства. Плата – некоторое увеличение сложности аппаратуры.

Квитирование не всегда производится в полном объёме. Иногда транзакция не может быть завершена стандартным образом (например обращение по несуществующему адресу памяти) – ведомое не ответит подтверждающим сигналом. Выход из ситуации – схема тайм-аута.

Тайм-ауты из-за отказа оборудования – редки, но тайм-ауты по адресу – часты (например, проверка присутствия устройств на шине) -> малые значения тайм-аута и очень быстрые схемы декод-я адреса.

Асинхронные обычно шины ввода/вывода.

Слайд 48

## Синхронные vs асинхронные

Ранее в ВМ преобладали асинхронные шины, в последних разработках – всё чаще синхронные.

В настоящее время синхронные чуть быстрее асинхронных и близки к макс. значениям распространения сигналов. Возможное ускорение асинхронных даст лишь незначительный выигрыш по сравн. с синхр-ми.

Перекося сигналов – в синхронных уже заложен в такт. частоту, в асинхронных должен учитываться в каждой транзакции и каждым устройством.

Проблема *метастабильного состояния* триггеров ведомых в асинхронных решается легче – спец. Схемы определяют подобные ситуации и асинхр. система ожидает пока состояние не станет стабильным. В синхронном протоколе – синхронизация триггеров ТИ и/или двухтактные триггеры.

Слайд 49

## Методы повышения эффективности шин

Приёмы повышения производительности шин:

- Пакетный режим
- Конвейеризация транзакций
- Расщепление транзакций

Пакетный или блочный режим (burst mode) – как для асинхр., так и для синхр-х. Один адресный цикл сопровождается множественными циклами данных (либо чтения либо записи, но не смешанных!). Пакет данных передаётся без указания адресов данных внутри пакета -> увеличение адреса производится автоматически как у ведущего, так и у ведомого. Длина пакета может достигать до 1024 байт.

## Методы повышения эффективности шин

Пакетный или блочный режим (burst mode) – наиболее частый вариант – передача пакетов из четырёх байт.



В асинхронных режимах – дополнительный выигрыш: разрешение отправителю начинать следующий цикл данных, не ожидая подтверждения от получателя -> экономия времени прохождения сигнала от отправителя к приёмнику, время подтверждения + внутренние задержки в ведущем и ведомом устр-вах и восстановления шины после квитирования.

Пример шины с пакетным режимом - FutureBus+.

Слайд 4

## Методы повышения эффективности шин

Конвейеризация транзакций: очередной элемент данных может быть послан устройством А, до того, как устройство В завершит считывание предыдущего элемента.



Данные должны оставаться стабильными в течение времени  $t_{ст}$  (стабилизации) +  $t_{уд}$  (удержания), только после этого – смена элемента данных.

Максимальная скорость передачи –  $1/(t_{ст} + t_{уд})$

## Методы повышения эффективности шин

Протокол с расщеплением транзакций (соединения/разъединения) или протокол с коммутацией пакетов (packet-switched): обеспечивает преимущество на транзакциях чтения.

В классическом варианте – любая транзакция непрерывна.

В протоколе с расщеплением – две шины ША и ШД обязательно разделены. Ведущий выставляет адрес и требование данных. После чего шина свободна. Когда память подготовит данные она становится ведущим и запрашивает доступ к шине, после чего пересылает данные запрашивающему. Соответственно, пока память готовит данные к отправке шина свободна и может использоваться другими ведущими...

## Методы повышения эффективности шин

Протокол с расщеплением транзакций – на такой шине имеет место поток запросов и поток откликов.

Контроллер памяти обычно проектируется так, чтобы обеспечивать буферизацию множественных потоков.

Возможен вариант организации шины с ответами на запросы в произвольном порядке (требуется вводить теги на данные).

Плюс – более эффективное использование шины.

Минусы – дополнительные подтверждения при запросе и отклике, дополнительные затраты на аппаратуру.

Для любой шины с расщеплением, существует предельное число одновременно обслуживаемых запросов.

## 26. Чем характеризуются ЗУ?

Память предназначена для фиксации, хранения и выдачи информации в процессе работы ЭВМ. Процессы чтения и записи информации определяются как процессы обращения к запоминающему устройству (ЗУ).

Перечень основных характеристик, которые необходимо учитывать, рассматривая конкретный вид ЗУ, включает в себя:

- месторасположения (процессорные, внутренние и внешние);
- емкость (число бит либо байт, которое может храниться в ЗУ);
- единицу пересылки (основной памяти (ОП) единица пересылки определяется шириной шины данных, т.е. количеством битов, передаваемых по линиям шины параллельно. Обычно единица пересылки равна длине слова, но не обязательно. Применительно к внешней памяти данные часто передаются единицами, превышающими размер слова, и такие единицы называются блоками);
- метод доступа (• Последовательный доступ. ЗУ с последовательным доступом ориентировано на хранение информации в виде последовательности блоков данных, называемых записями. Для доступа к нужному элементу (слову или байту) необходимо прочитать все предшествующие ему данные. • Прямой доступ. Каждая запись имеет уникальный адрес, отражающий ее физическое размещение на носителе информации. Обращение осуществляется как адресный доступ к началу записи, с последующим последовательным доступом к определенной единице информации внутри записи. • Произвольный доступ. Каждая ячейка памяти имеет уникальный физический адрес. Обращение к любой ячейке занимает одно и то же время и может вестись в произвольной очередности. • Ассоциативный доступ. Этот вид доступа позволяет выполнять поиск ячеек, содержащих такую информацию, в которой значение отдельных битов совпадает с состоянием одноименных битов в заданном образце);
- быстродействие (• Время доступа. Для памяти с произвольным доступом оно соответствует интервалу времени от момента поступления адреса до момента, когда данные заносятся в память или становятся доступными. В ЗУ с подвижным носителем информации - это время, затрачиваемое на установку головки записи/считывания (или носителя) в нужную позицию. • Длительность цикла памяти или период обращения. Понятие применяется к памяти с произвольным доступом, для которой оно означает минимальное время между двумя последовательными обращениями к памяти. Период обращения включает в себя время доступа плюс некоторое дополнительное время. • Скорость передачи - скорость, с которой данные могут передаваться в память или из нее);
- физический тип (полупроводниковая память, память с магнитным носителем информации, память с оптическим носителем);
- физические особенности (энергозависимость, приводит ли считывание информации к ее разрушению);
- стоимость (отношением общей стоимости ЗУ к его емкости в битах, т.е. стоимостью хранения одного бита информации.).

## 27. Основная проблема памяти и способ её решения.

Память часто называют «узким местом» фон-неймановских ВМ из-за ее серьезного отставания по быстродействию от процессоров, причем разрыв этот неуклонно увеличивается. Так, если производительность процессоров ежегодно возрастает вдвое примерно каждые 1,5 года, то для микросхем памяти прирост быстродействия не превышает 9% в год (удвоение за 10 лет), что выражается в увеличении разрыва в быстродействии между процессором и памятью приблизительно на 50% в год.

Если проанализировать используемые в настоящее время типы ЗУ, выявляется следующая закономерность:

- чем меньше время доступа, тем выше стоимость хранения бита;

- чем больше емкость, тем ниже стоимость хранения бита, но больше время доступа.

При создании системы памяти постоянно приходится решать задачу обеспечения требуемой емкости и высокого быстродействия за приемлемую цену. Наиболее распространенным подходом здесь является построение системы памяти ВМ по иерархическому принципу. Иерархическая память состоит из ЗУ различных типов, которые, в зависимости от характеристик, относят к определенному уровню иерархии. Более высокий уровень меньше по емкости, быстрее и имеет большую стоимость в пересчете на бит, чем более низкий уровень. Уровни иерархии взаимосвязаны: все данные на одном уровне могут быть также найдены на более низком уровне, и все данные на этом более низком уровне могут быть найдены на следующем нижележащем уровне и т. д.

Четыре верхних уровня иерархии образуют внутреннюю память ВМ, а все нижние уровни - это внешняя или вторичная память.

По мере движения вниз по иерархической структуре:

1. Уменьшается соотношение «стоимость/бит».
2. Возрастает емкость.
3. Растет время доступа.
4. Уменьшается частота обращения к памяти со стороны центрального процессора

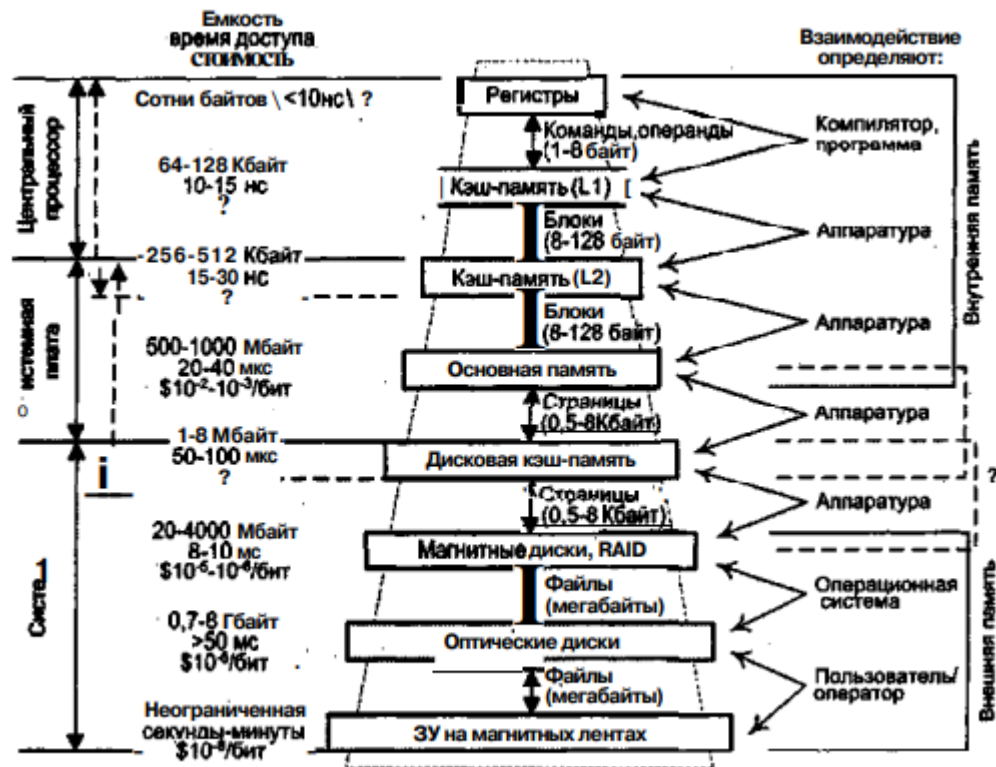


Рис. 5.1. Иерархия запоминающих устройств

## 28. Система ввода-вывода, понятие и задачи.

Система ввода/вывода должна обеспечивать:

- построение вычислительной системы с переменным составом оборудования
- реализацию параллельной работы процессора над программой и выполнение им процедур ввода/вывода
- простоту и стандартность операций ввода/вывода, обеспечивая независимость программ от особенностей ПУ
- автоматическое распознавание и обслуживание ПУ

Способы обмена:

- программный (не форсированный)

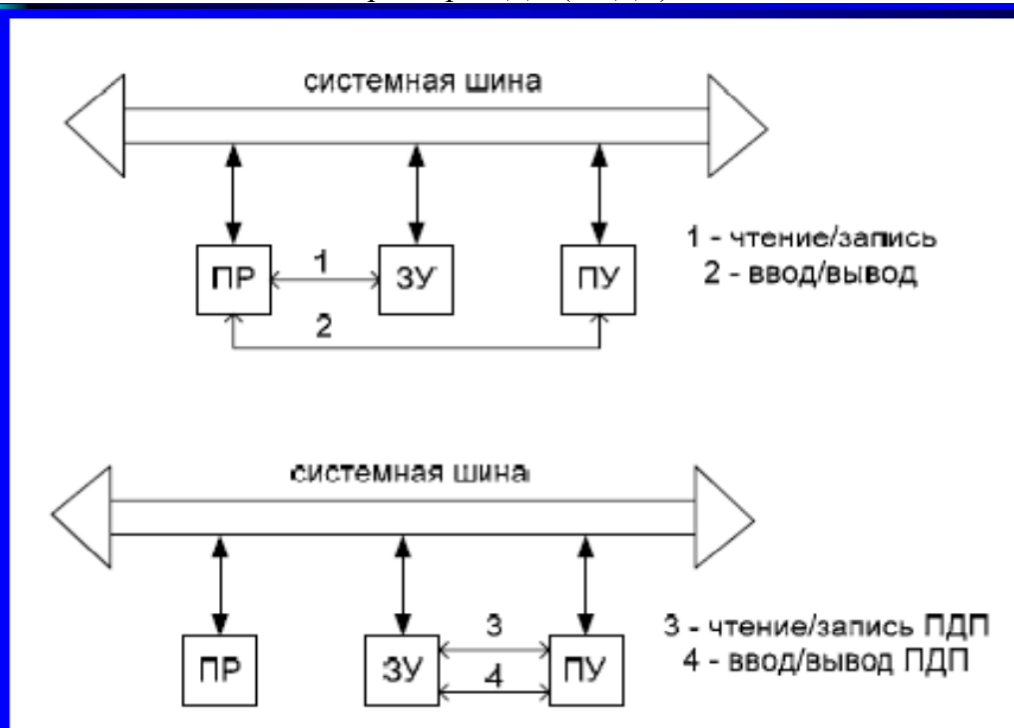
- по прерыванию (форсированный)
- при прямом доступе к памяти (аппаратный)

Программно управляемый обмен данными осуществляется по инициативе процессора и под его управлением. Данные между процессором и памятью, памятью и внешним устройством пересылаются через процессор (регистр АХ). При таком обмене процессор на всё время его выполнения отвлекается от других команд – снижается производительность ЭВМ. Пересылая блок данных ЦП выполняет много вспомогательных действий:

- буферизация данных
- преобразование форматов данных
- подсчёт количества передач
- учёт ограничений по счётчику
- формирование адреса памяти
- обнаружение конечных символов при передаче цепочек

В результате скорость передачи данных снижается в десятки раз

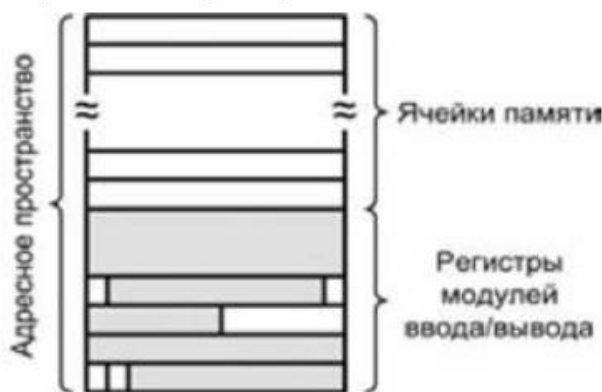
Форсировать скорость передачи данных с ПУ можно используя обмен по прерываниям. В этом случае обмен осуществляется по требованию ПУ, когда оно готово к нему, а идентификацию устройства производить по вектору прерывания, переходя соответственно к программе обслуживания ПУ. Это всё повышает скорость передачи, но принцип программно управляемого обмена остаётся неизменным. Максимально быстрый режим обмена данными осуществляется с помощью ПДП (прямой доступ к памяти). ПДП происходит без участия процессора, данные перекачиваются в ПУ непосредственно из памяти (минуя процессор) и наоборот (аппаратная передача). Этот режим реализуется с помощью дополнительного контроллера ПДП (КПДП)



Прямой доступ к памяти (мимо процессора)

## 29. Совмещенное адресное пространство СВВ, достоинства и недостатки.

При совмещении адресного пространства для адресации модулей ввода/вывода отводится определенная область адресов. Обычно все операции с модулем ввода/вывода осуществляются с использованием входящих в него внутренних регистров: управления, состояния, данных. Фактически процедура ввода/вывода сводится к записи информации в одни регистры МВВ и считыванию ее из других регистров. Это позволяет рассматривать регистры МВВ как ячейки основной памяти и работать с ними с помощью обычных команд обращения к памяти, при этом в системе команд ВМ вообще могут отсутствовать специальные команды ввода и вывода. Так, модификацию регистров МВВ можно производить непосредственно с помощью арифметических и логических команд. Адреса регистрам МВВ назначаются в области адресного пространства памяти, отведенной под систему ввода/вывода.



Достоинства совмещенного адресного пространства:

- расширение набора команд для обращения к внешним устройствам, что позволяет сократить длину программы и повысить быстродействие;
- значительное увеличение количества подключаемых внешних устройств;
- возможность внепроцессорного обмена данными между внешними устройствами, если в системе команд есть команды пересылки между ячейками памяти;
- возможность обмена информацией не только с аккумулятором, но и с любым регистром центрального процессора.

Недостатки совмещенного адресного пространства:

- сокращение области адресного пространства памяти;
- усложнение декодирующих схем адресов в СВВ;
- трудности распознавания операций передачи информации при вводе/выводе среди других операций. Сложности в чтении и отладке программы, в которой простые команды вызывают выполнение сложных операций ввода/вывода;
- трудности при построении СВВ на простых модулях ввода/вывода: сигналы управления не смогут координировать сложную процедуру ввода/вывода. Поэтому МВВ часто должны генерировать дополнительные сигналы под управлением программы.

## 30. Конфликты в конвейере команд. Типы рисков.

Идея конвейера команд была предложена в 1956 году академиком С. А. Лебедевым. Как известно, цикл команды представляет собой последовательность этапов. Возложив реализацию каждого из них на самостоятельное устройство и последовательно соединив такие устройства, мы получим классическую схему конвейера команд.

Для иллюстрации воспользуемся примером. Выделим в цикле команды шесть этапов:

1. Выборка команды. Чтение команды из памяти и занесение ее в регистр.

2. Декодирование команды. Определение кода операции и способов адресации операндов.

3. Вычисление адресов операндов (ВА). Вычисление исполнительных адресов каждого из операндов в соответствии с указанным в команде способом их адресаций.

4. Выборка операндов (ВО). Извлечение операндов из памяти. Эта операция не нужна для операндов, находящихся в регистрах.

5. Исполнение команды (ИК). Исполнение указанной операции.

6. Запись результата (ЗР), Занесение результата в память.

Конфликтные ситуации в конвейере принято обозначать термином риск (hazard), а обусловлены они могут быть тремя причинами:

- попыткой нескольких команд одновременно обратиться к одному и тому же ресурсу ВМ, чаще всего память (структурный риск) (подобных конфликтов частично удается избежать за счет модульного построения памяти и использования кэш-памяти - имеется вероятность того, что команды будут обращаться либо к разным модулям ОП, либо одна из них станет обращаться к основной памяти, а другая - к кэш-памяти. С этих позиций выгоднее разделять кэш-память команд и кэш-память данных.);

- взаимосвязью команд по данным (риск по данным) (- «Чтение после записи» (ЧПЗ): команда j читает x до того, как команда i успела записать новое значение x, т.е. j ошибочно получит старое значение x вместо нового. - «Запись после чтения» (ЗПЧ): команда j записывает новое значение x до того, как команда i успела прочитать x, т.е. команда i ошибочно получит новое значение x вместо старого. - «Запись после записи» (ЗПЗ): команда j записывает новое значение x прежде, чем команда i успела записать в качестве x свое значение, т.е. x ошибочно содержит i-е значение x вместо j- го);

- неоднозначностью при выборке следующей команды в случае команд перехода (риск по управлению) (любые переходы кроме jmp).

## Конфликты в конвейере команд

Устранение конфликтов – программно и аппаратно:

Программно – компиляция оптимизирующим компилятором (в спорных случаях ставится NOP).

Аппаратно – пузырьёк и ускоренное продвижение информации (Forwarding).

Пузырёк – приостановка на несколько тактов команды j, чтобы команда i успела завершиться или пройти конфликтную ступень.

Задерживаются и все команды, следующие за командой j. Иногда приостанавливают j, но не задерживают остальные команды – эффективно, но требует сложной реализации конвейера.

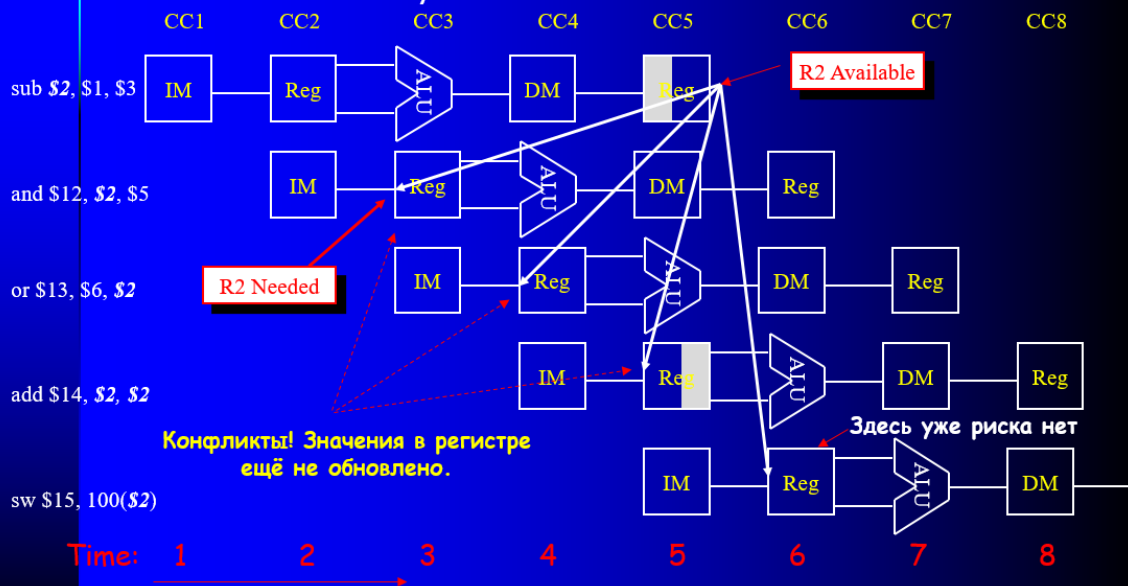
Forwarding - обычно буферный регистр между двумя ступенями конвейера, передача информации возможна только между соседними ступенями конвейера. При forwarding, операнд уже вычисленный предыдущей командой может быть получен непосредственно из соотв. буфера, минуя все промежуточные ступени конвейера -> доп. тракты данных (опережения, перехода) + соответствующее усложнение УУ.

Слайд 4

### 31. Риск по данным, типы конфликтов по данным.

# Data Hazards

Риск по данным ("data hazard") – если результат требуется до того как он получен.



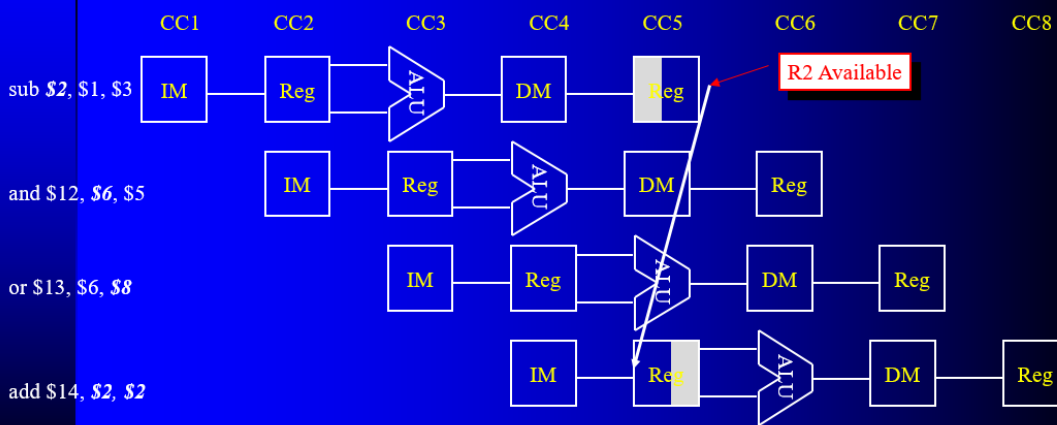
Слайд 8

## Устранение риска по данным

## Введение «прозрачного» набора регистров:

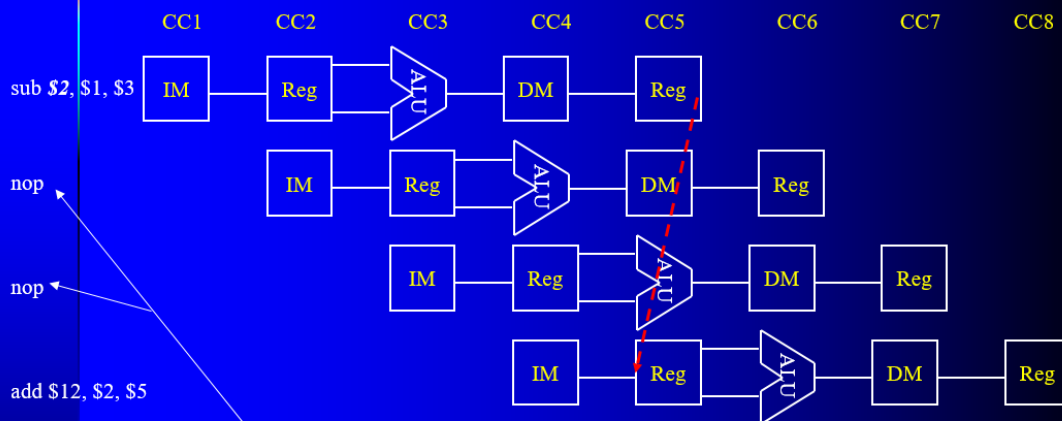
## Использование защёлок (latches) вместо триггеров (flip-flops) в регистрах

- Первую половину цикла 5: регистр 2 загружается
- Вторую половину цикла: новое значение считывается



Слайд 9

## Программный подход устранения риска по данным



Вставка достаточного количества «no-ops» ( или других команд, не обращающихся к регистру 2) устраняет риск по данным

Слайд 10

## За и против (Pros/Cons)

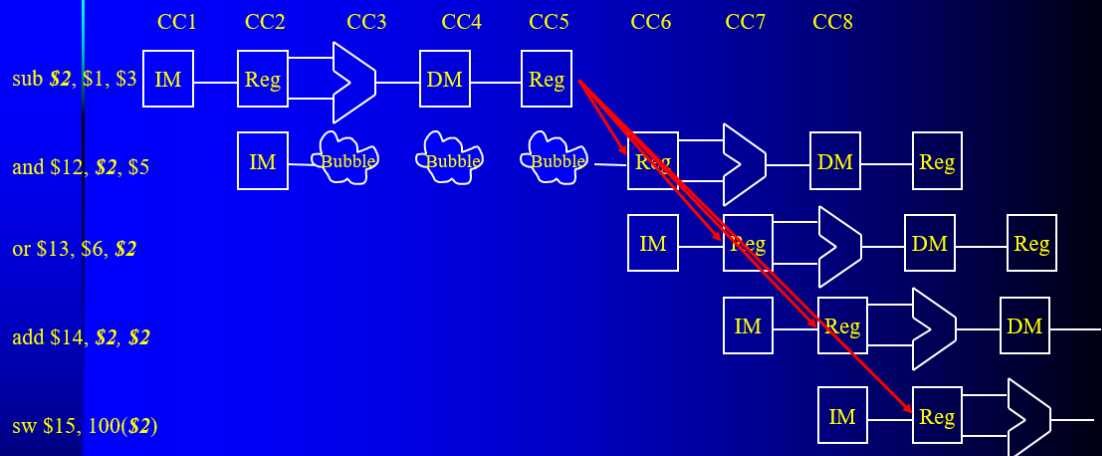
- Нет дополнительной аппаратуры – проблема передаётся на откуп компилятору
- В худшем случае требует «разбавления» кода «водой»
- Увеличение размера кода
- Сложности с поддержкой совместимости
  - Правила для отслеживания в командах риска изменяются при изменении конфигурации конвейера

Последний минус – самый главный «гвоздь в крышку гроба»

- Требование производителей полной смены программ при апгрейде системы очень не понравится покупателям.

Слайд 11

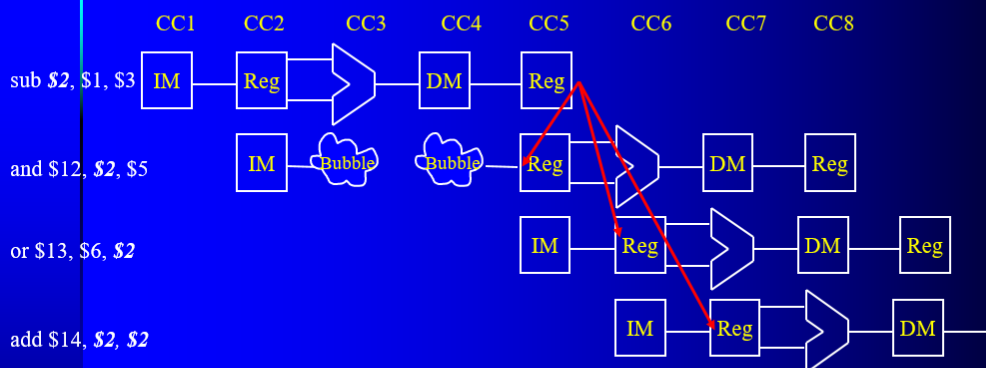
## Пузырёк – приостановка конвейера Pipeline Stalls



Приостановка в случае реализации блока регистров на триггерах.

Слайд 12

## Latches instead of flip-flops

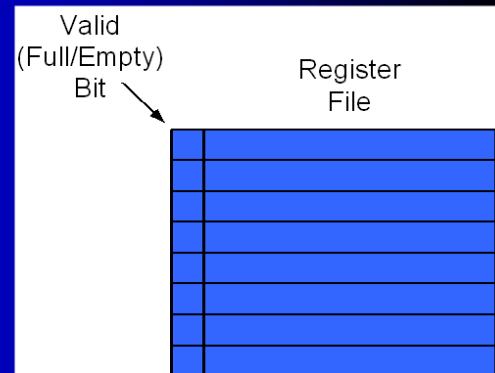


Приостановка в случае реализации блока регистров на защёлках.

Слайд 13

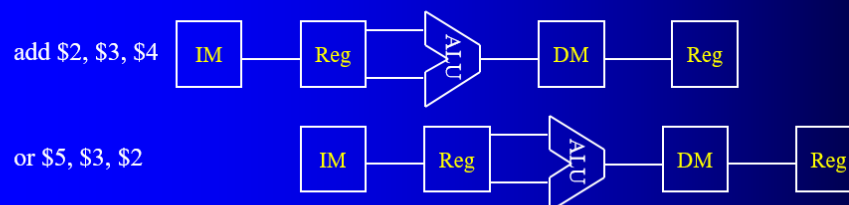
## Флаг доступности данных. Register Scoreboard – Tracking Operand Availability

- К каждому регистру из пула регистров добавляется по 1 биту.
- Аппаратура сбрасывает флаг для соотв. регистра, если в конвейере появляется команда, записывающая результат в регистр.
- Флаг устанавливается в 1, если команда завершилась и записала результат в соотв. регистр.
- Команды не допускаются к выполнению до тех пор, пока их регистры – источники операндов не станут доступными

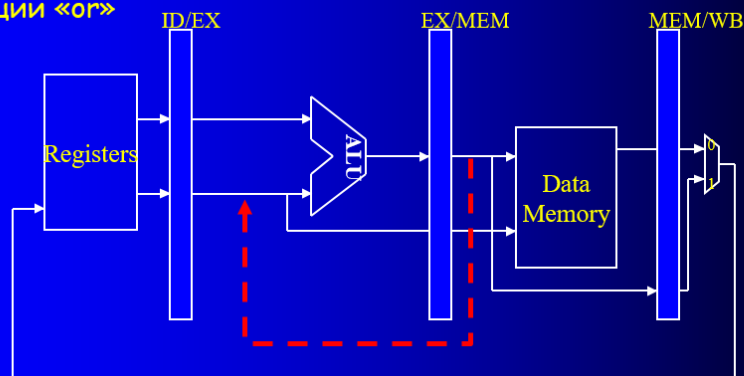


Слайд 14

## Устранение риска по данным с помощью Forwarding



Можно избежать остановки конвейера, если предусмотреть возможность передачи результата операции «add» с выхода ALU прямо на его вход для операции «or»



Слайд 15

## Ускоренное продвижение информации Data Forwarding

- Forwarding устраняет риск по данным на стадиях:
  - Выполнения команды (EX hazard)
  - Записи/чтения в/из памяти (MEM hazard)
- Использование «прозрачного» пула регистров позволяет бороться с риском по данным на третьей стадии – записи результата (WB)
  - в случае, если пул регистров запрашивается на одновременное чтение и запись из одного регистра в один цикл, то «прозрачный» пул регистров позволяет перенаправить записываемые данные прямо на выход.

Слайд 16

Типы конфликтов по данным:

1) Конфликты типа RAW (**Read After Write**): команда  $j$  пытается прочитать операнд прежде, чем команда  $i$  запишет на это место свой результат. При этом команда  $j$  может получить некорректное старое значение операнда.

Конфликты типа RAW обусловлены именно конвейерной организацией обработки команд.

Главной причиной двух других типов конфликтов по данным является возможность неупорядоченного выполнения команд в современных микропроцессорах, то есть выполнение команд не в том порядке, в котором они записаны в программе.

2) Конфликты типа WAR (**Write After Read**): команда  $j$  пытается записать результат в приемник, прежде чем он считается оттуда командой  $i$ . При этом команда  $i$  может получить некорректное новое значение операнда:

- 3)    i) ADD R1,R2  
       $i+1 = j$ ) SUB R2,R3

Этот конфликт возникнет в случае, если команда  $j$  вследствие неупорядоченного выполнения завершится раньше, чем команда  $i$  прочтает старое содержимое регистра R2.

4) Конфликты типа WAW (**Write After Write**): команда  $j$  пытается записать результат в приемник, прежде чем в этот же приемник будет записан результат выполнения команды  $i$ , то есть запись заканчивается в неверном порядке, оставляя в приемнике результата значение, записанное командой  $i$ .

### 32. Предсказание переходов — статический и динамический.

Предсказание переходов на сегодняшний день рассматривается как один из наиболее эффективных способов борьбы с конфликтами по управлению. Идея заключается в том, что еще до момента выполнения команды условного перехода или сразу же после ее поступления на конвейер делается предположение о наиболее вероятном исходе такой команды (переход произойдет или не произойдет). Последующие команды подаются на конвейер в соответствии с предсказанием. Если предсказан переход, то после команды  $x$  на конвейер подаются команды  $y$ ,  $y+1$ , ... При ошибочном предсказании конвейер необходимо вернуть к состоянию, с которого началась выборка

«ненужных» команд (очистить начальные ступени конвейера), и приступить к загрузке, начиная с «правильной» точки, что по эффекту эквивалентно приостановке конвейера.

Цена ошибки может оказаться достаточно высокой, но при правильных предсказаниях крупен и выигрыш — конвейер функционирует ритмично без остановок и задержек, причем выигрыш тем больше, чем выше точность предсказания. Термин «точность предсказания» в дальнейшем будем трактовать как процентное отношение числа правильных предсказаний к их общему количеству. В работе [70] дается следующая оценка: чтобы снижение производительности конвейера из-за его остановок по причине конфликтов по управлению не превысило 10%, точность предсказания переходов должна быть выше 97,7%.

Статическое предсказание переходов осуществляется на основе некоторой априорной информации о подлежащей выполнению программе. Предсказание делается на этапе компиляции программы и в процессе вычислений уже не меняется. Главное различие между известными механизмами статического прогнозирования заключается в виде информации, используемой для предсказания, и ее трактовке. Исходная информация может быть получена двумя путями: на основе анализа кода программы или в результате ее профилирования. Известные стратегии статического предсказания для команд УП можно классифицировать следующим образом:

- переход происходит всегда (ПВ);
- переход не происходит никогда (ПН);
- предсказание определяется по результатам профилирования;
- предсказание определяется кодом операции команды перехода;
- предсказание зависит от направления перехода;

В динамических стратегиях решение о наиболее вероятном исходе команды УП принимается в ходе вычислений, исходя из информации о предшествующих переходах (истории переходов), собираемой в процессе выполнения программы. В целом, динамические стратегии по сравнению со статическими обеспечивают более высокую точность предсказания. Идея динамического предсказания переходов предполагает накопление информации об исходе предшествующих команд УП. История переходов фиксируется в форме таблицы, каждый элемент которой состоит из  $t$  битов. Нужный элемент таблицы указывается с помощью  $k$ -разрядной двоичной комбинации - шаблона (pattern). Этим объясняется общепринятое название таблицы предыстории переходов - таблица истории для шаблонов (PHT, Pattern History Table).

В качестве шаблонов для доступа PHT могут быть: адрес команды условного перехода, регистр глобальной истории, регистр локальной истории, комбинация этих вариантов.

При описании динамических схем предсказания переходов их часто расценивают как один из видов автоматов Мура, при этом содержимое элементов PHT трактуется как информация, отображающая текущее состояние автомата. Существуют различные варианты подобных автоматов, однако реально осмысленно говорить лишь о трех вариантах: A1, A2 и A3. Автоматы могут принимать 4 состояния (00 – перехода нет сильн., 01 – перехода нет слаб., 10 – переход есть слаб., 11 – переход есть сильн.).

### **33. Стратегии статического перехода.**

Статическое предсказание переходов осуществляется на основе некоторой априорной информации о подлежащей выполнению программе. Предсказание делается на этапе компиляции программы и в процессе вычислений уже не меняется. Главное различие между известными механизмами статического прогнозирования заключается в виде информации, используемой для предсказания, и ее трактовке. Исходная информация может быть получена двумя путями: на основе анализа кода программы или в результате ее профилирования. Известные стратегии статического предсказания для команд УП можно классифицировать следующим образом:

- переход происходит всегда (ПВ);
- переход не происходит никогда (ПН);
- предсказание определяется по результатам профилирования;
- предсказание определяется кодом операции команды перехода;
- предсказание зависит от направления перехода;

В первом из перечисленных вариантов предполагается, что каждая команда условного перехода в программе обязательно завершится переходом, и, с учетом такого предсказания, дальнейшая выборка команд производится, начиная с адреса перехода. В основе второй стратегии лежит прямо противоположный подход: ни одна из команд условного перехода в программе никогда не завершается переходом, поэтому выборка команд продолжается в естественной последовательности.

Цифры свидетельствуют, что успешность стратегии ПВ (в среднем >50%) существенно зависит от характера программы и методов программирования, что, естественно, можно рассматривать как недостаток.

Ни ПВ, ни ПН не обладает явным преимуществом над другой. Тем не менее анализ большого количества программ показывает, что условные переходы имеют место более чем в 50% случаев, поэтому если стоимость реализации двух рассмотренных вариантов одинакова, то предпочтение следует отдать стратегии ПВ.

В третьем из перечисленных способов статического предсказания назначение командам УП наиболее вероятного исхода производится по результатам профилирования подлежащих выполнению программ, т.е. выполнялась программа при некотором эталонном наборе исходных данных, сопровождающееся сбором информации об исходах каждой команды условного перехода. Тем командам, которые чаще завершались переходом, назначается стратегия ПВ, а всем остальным - ПН. Выбор фиксируется в специальном бите кода операции. Некоторые компиляторы самостоятельно проводят профилирование программы и по его результатам устанавливают этот бит в формируемом объектном коде. При выполнении программы поведение конвейера команд определяется после выборки команды по состоянию упомянутого бита в коде операции. Основным недостатком этого образа действий связан с тем, что изменение набора исходных данных для профилирования может существенно менять поведение одних и тех же команд условного перехода.

При предсказании на основе кода операции команды перехода для одних команд предполагается, что переход произойдет, для других - его не случится. Например, для команды перехода по переполнению логично предположить, что перехода не будет. На практике назначение разным видам команд УП наиболее вероятного исхода чаще производится по результатам профилирования программ.

Достаточно логичным представляется предсказание исхода из направления перехода. Если указанный в команде адрес перехода меньше содержимого счетчика команд, говорят о переходе «назад», и для такой команды назначается стратегия ПВ. Переход к адресу, превышающему адрес команды перехода, считается переходом «вперед», и такой команде назначается стратегия ПН. В основе рассматриваемого подхода лежит статистика по множеству программ, согласно которой большинство команд УП в программах используются для организации циклов, причем, как правило, переходы происходят к началу цикла (переходы «назад»).

В последней из рассматриваемых статических стратегий при первом выполнении любой команды условного перехода делается предсказание, что переход обязательно будет. Предсказания на последующее выполнение команды зависят от правильности начального предсказания. Стратегию можно считать статической только частично, поскольку она однозначно определяет исход команды лишь при первом ее выполнении. Точность прогноза в соответствии с этой стратегией выше, чем у всех предшествующих.

## 34. Классификация архитектур системы команды (АСК)

Архитектура системы команд – интерфейс между всем выполняемым на машине ПО и аппаратным обеспечением.

АСК(ISA) характеризуется ответами на вопросы:

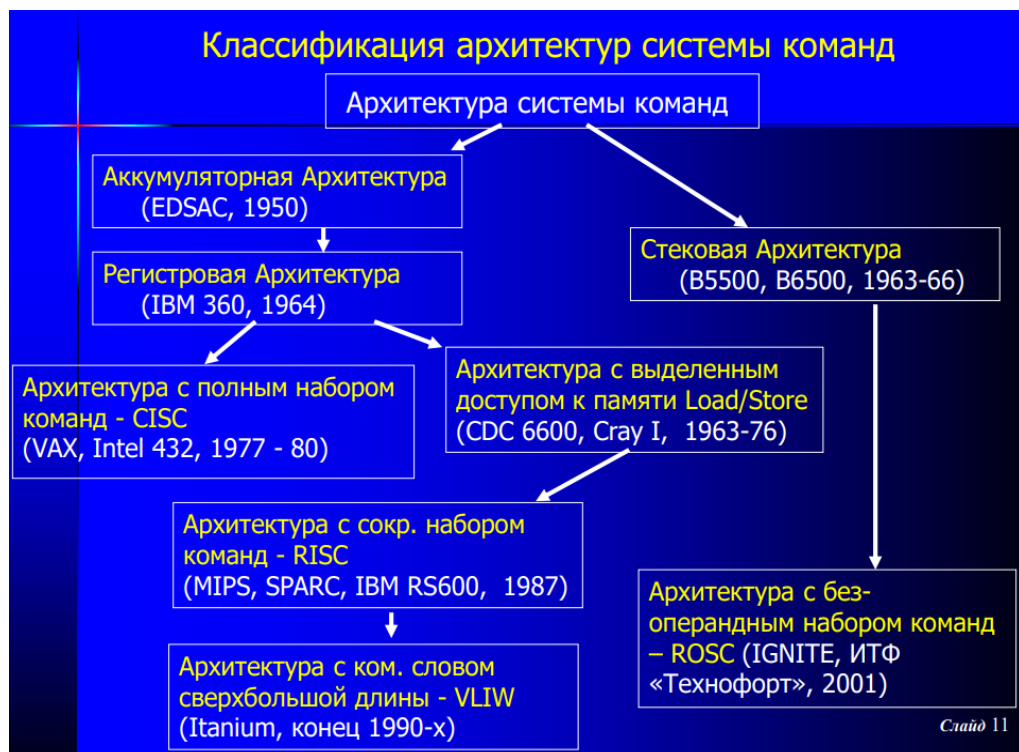
- Какого вида данные будут представлены в ВМ и в какой форме?
- Где эти данные могут храниться помимо основной памяти?
- Каким образом будет осуществляться доступ к данным?
- Какие операции могут быть выполнены над данными?
- Сколько операндов может присутствовать в команде?
- Как будет определяться адрес очередной команды?
- Каким образом будут закодированы команды?

АСК (ISA) – очень важный уровень абстракции:

- интерфейс между аппаратурой и низко-уровневым ПО
- стандартизирует команды, шаблоны битов машинного языка и т.д.
- преимущества: позволяет реализовывать различные аппаратные варианты одной программной архитектуры
- недостатки: периодически препятствует внедрению новых технологий и прочих инноваций

Современные АСК: – 80x86/Pentium/K6, PowerPC, DEC Alpha, MIPS, SPARC,

HP



Классификация по месту хранения команд:

- **Аккумуляторная архитектура (EDSAC, 1950)**

Исторически первая архитектура. В процессоре выделенный регистр – аккумулятор. В этот регистр – заносится результат операции. Изначально оба операнда в памяти. Перед операцией один из них – в аккумулятор. После выполнения результат в аккумуляторе (либо в памяти - если он не является операндом для следующей команды).

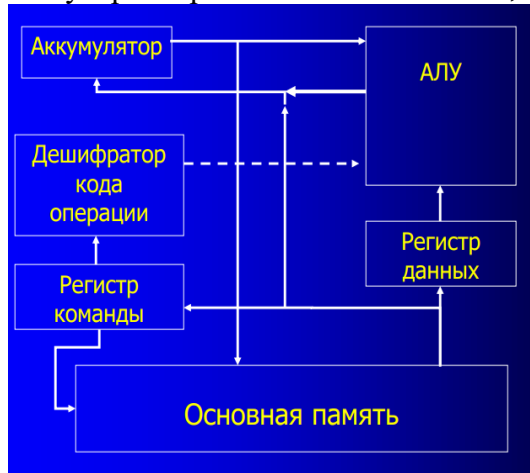
Достоинства:

- Короткие команды
- Простота декодирования команд

Недостатки:

- многократные обращения к памяти

Популярна в ранних ВМ – IBM 7090, DEC PDP-8, MOS 6502



- **Стековая архитектура (B5500, B6500, 1963-66)**

Стек – множество логически взаимосвязанных по принципу LIFO ячеек. Две операции – push и pop. Запись и чтение только в/из вершины стека. Операнды помещаются в стек, в процессе операции они выталкиваются из стека. Результат операции – в стек. Для стековой архитектуры лучше всего подходит обратная польская нотация (Чарльз Хэмблин, 1957), разработанная на основе «польской нотации» (Я. Лукасевич, 1920)

Достоинства:

- стек может быть неоднороден (верхние ячейки быстрые, нижние медленные).
- сокращение адр. части команд.
- компактный код
- простой компилятор
- простое декодирование команд

Недостатки:

Компилятор не может создать эф. код (нет произвольного обращения к памяти).

Стек – узкое место.

Низкая производительность.

Расцвет архитектуры – 60е годы: HP 2116B, HP 3000/70. Последние годы – повышение интереса к стековой архитектуре. Удобен для Java и Forth. Машины – JEM 1 и JEM 2 от aJile Systems, Clip от Imsys. ROSC (Removed Operand Set Computer) – новая реинкарнация стековой архитектуры. IGNITE от Patriot Scientist



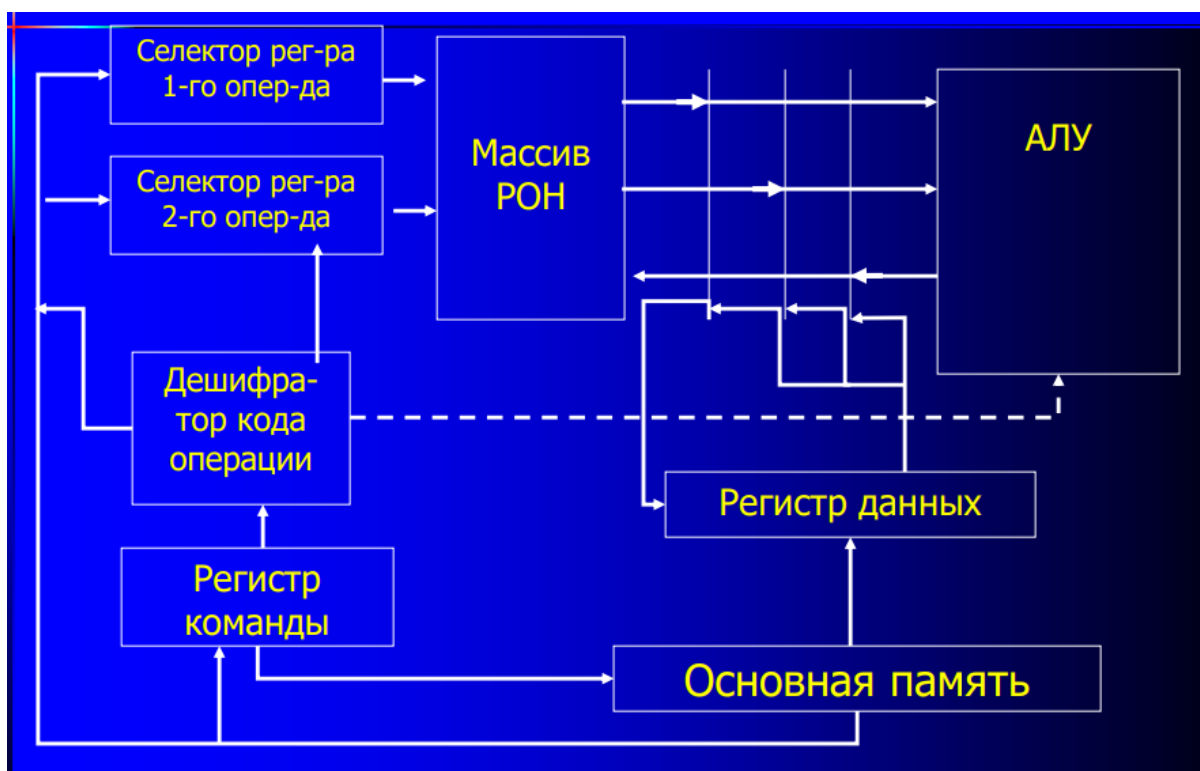
- **Регистровая Архитектура (IBM 360, 1964)**

Массив регистров (РОН) – явно управляемый для хранения недавно использовавшихся данных. Размер регистров – обычно совпадает с маш. словом. К люб. регистру можно обратиться по номеру. Меньшее количество регистров – меньше разрядов на кодирование

номера регистра – короче команда. Операнды – либо в памяти либо в регистрах.

Три подвида команд: Регистр-регистр, Регистр-память, Память-память

| Вариант         | Достоинства                                                                                                                        | Недостатки                                                                                                                                 |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Регистр-регистр | Простота реализации, фикс. длина команд, простота кода при компиляции, возможность выполнения всех команд за одинак. кол-во тактов | Большая длина кода, из-за фикс. длины команд – часть разрядов в команде не используется.                                                   |
| Регистр-память  | Данные могут быть доступны и без загрузки в рег., простота кодирования команд, компактный код                                      | Потеря одного из опер. при записи результата, длинное поле адреса в команде – сокращает кол-во РОН, CPI зависит от места размещ. операнда. |
| Память-память   | Компактность кода, малая потребность в регистрах для хранения промежут. результатов                                                | Разнообразие форматов команд и времени их исполнения, низкое быстродействие.                                                               |



Операции загрузки операндов в РОН – идентичны работе с аккумулятором. Отличие – выбор нужного регистра. Выполнение операции АЛУ включает в себя: Выбор регистра первого операнда Определение местоположения второго оперда (память или регистр) Подача на вход АЛУ операндов и выполнение операции Выбор регистра результата и занесение в него результата операции из АЛУ.

Достоинства – компактность кода, высокая скорость.

Недостатки – более длинные инструкции (по сравнению с аккумуляторной архитектурой)

Регистровая АСК – преобладающая в настоящее время (все современные персоналки).

Первые машины – IBM 360/370, PDP-11

• **Архитектура с выделенным доступом к памяти Load/Store (CDC 6600, Cray I, 1963-76)**

Загрузка в регистр N ячейки x – load x N

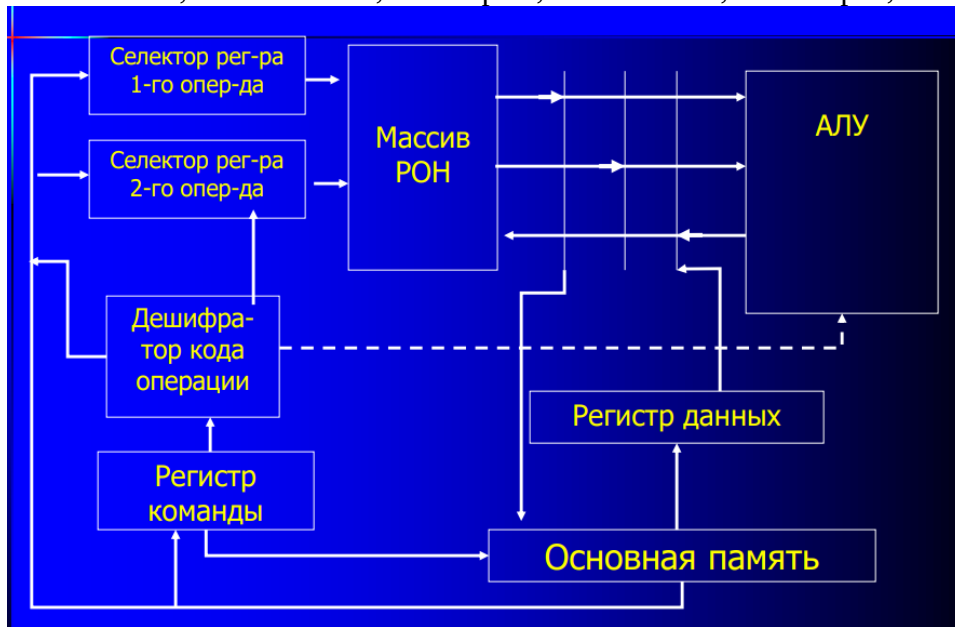
Запись из регистра N в ячейку x – store N x

Данная АСК характерна для всех ВМ с RISC архитектурой. Команды как правило – трёх

адресный формат и 32 бита длина.

Достоинства Load/Store ACK – простота декодирования и исполнения команд.

HP PA-RISC, IBM RS/6000, SUN Sparc, MIPS R4000, DEC Alpha, etc.



### 35. Принципы концепции ВМ фон Неймана. Какой из принципов можно назвать наиболее существенным?

Принципы ВМ Фон Неймана:

- **принцип хранимой в памяти программы**
  - ENIAC – (1946 -1955), программа задавалась схемой коммутации триггеров на 40 наборных полях.
  - EDVAC – (1949-...) первая машина с хранимой в памяти программой
- **двоичное кодирование**

Вся информация (команды, данные) – в бинарном виде

  - Каждый тип информации имеет свой формат
  - Последовательность битов, связанных одним смыслом, называется полем:
    - для данных
    - поле знака и поле значащих разрядов
    - для команды
    - поле кода операции и поле адресов
- **линейное пространство памяти (адресность)**
  - Память – набор пронумерованных ячеек
  - В каждый момент доступна любая ячейка
  - Двоичные коды данных и команд хранятся в единицах информации – словах
  - Доступ к данным по номерам ячеек – адресам
- **однородность памяти (безразличие к целевому назначению данных) Принстон**
  - Команды и данные хранятся в одной памяти и внешне не различимы
  - Возможность производить над командами те же операции, что и над данными (модификация команд)
  - Модификация команд современным программированием не приветствуется
  - Самый полезный эффект данного принципа – команды одной программы могут быть получены как результат работы другой программы. Возможность трансляции – главное следствие.
  - Команды и данные хранятся в одной памяти и внешне не различимы – Принстонская архитектура
  - Гарвардская архитектура – отдельная память команд и отдельная память данных.

- Узкое место принстонской архитектуры – пропускная способность тракта «процессор-память»
- В последнее время благодаря -памяти всё чаще начинает использоваться гарвардская архитектура.
- **программное управление (последовательное выполнение команд)**
  - Все вычисления алгоритма должны быть представлены в виде программы
  - Программа состоит из последовательности управляющих слов - команд
  - Команды хранятся в последовательных ячейках памяти и выполняются в естественной последовательности
  - Последовательность может быть изменена – условно или безусловно



Рис. 1.3. Структура фон-неймановской вычислительной машины

## 36. Принципы EPIC архитектуры (IA-64)

Использование архитектуры EPIC - главное отличие Itanium от RISC-процессоров. Она позволяет разбивать различные приложения и процессы вычислений на выполняемые параллельно фрагменты, обеспечивая в результате существенный скачок производительности. В архитектуре EPIC сочетаются многие технологические решения, которые в результате сочетания дают значительное повышение скорости обработки и решение некоторых проблем трансляции программ. Рассмотрим эти решения:

- Поддержка явно выделенного компилятором параллелизма.
- Большой регистровый файл.
- Подобно архитектуре PowerPC, в EPIC присутствуют предикатные регистры.

**Использование предикатных регистров** – важнейшая особенность, кардинально отличающая IA-64 от всех других процессоров.

- Спекулятивная загрузка данных, позволяющая избежать простоев конвейера при загрузке данных из оперативной памяти

- Поддержка предикатно выполняемых команд, которая позволяет избежать лишних инструкций ветвления, если количество команд в ветвях условного оператора невелико и уменьшить нагрузку на устройство предсказания ветвлений.

- Аппаратная поддержка программной конвейеризации с помощью механизма переименования регистров.

- Используется стек регистров (и регистровые окна).
- Для предсказания инструкций используется поддержка компилятора.
- Введена также поддержка инструкций циклического выполнения команд без потерь времени на инструкции циклического выполнения.

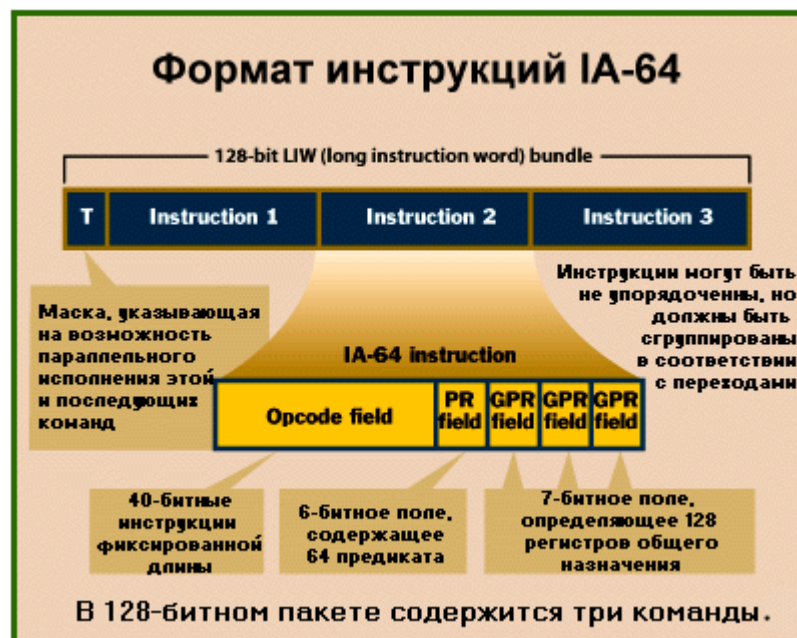
В EPIC, в отличие от RISC, задача распараллеливания инструкций возложена на компилятор. Это позволяет упростить структуру процессора, и как следствие, добиться более высокой его производительности при меньшей степени сложности. Кроме

**того, компилятор оптимизирует выполнение программы целиком, а не отдельного ее фрагмента, который доступен процессору в случае аппаратного распараллеливания.**

Наиболее интересной в архитектуре EPIC является поддержка спекулятивного выполнения команд и загрузка данных. Рассмотрим их подробнее.

Большинство команд загрузки данных из памяти выполняется длительное время. Выполнение команды за 1-2 такта возможно только в том случае, если значение содержится в -памяти второго уровня. Время несколько увеличивается, если значение находится в -памяти второго уровня. В случае чтения же данных из микросхем динамической памяти даже при попадании на активную страницу происходит значительная задержка при загрузке, а при смене страницы задержка имеет огромную величину, причём конвейер быстро блокируется, так команд, которые можно выполнить без нарушения зависимости по данным, обычно оказывается крайне мало. При спекулятивном выполнении используется вынесение команд загрузки далеко вперёд инструкций, использующих эти данные, в основном вверх за инструкции условного перехода. При достижении потоком команд места, где необходимо использовать загружаемые данные, вставляется инструкция, проверяющая, не произошло ли исключения в процессе загрузки (например, какая-то инструкция произвела запись по этому адресу), если исключение произошло, то вызывается специально написанный восстановительный код, который попросту перезагружает значение в регистр. Также при спекулятивных операциях по данным оптимизируется часто встречающийся участок во многих программах





В двух пространственных интервью журналу BYTE они раскрыли некоторые детали архитектуры IA-64.

- Команды в формате IA-64 упакованы по три в 128-битный пакет для быстрой обработки. Обычно это называют "LIW encoding". (Русский аналог подобрать сложно. Наиболее адекватно, на мой взгляд, перевести как "кодирование в длинные слова команд".) Однако компания Intel избегает такого названия, заявляя, что с ним связаны "негативные ассоциации" (negative connotation). По той же причине Intel не любит называть сами команды RISC-подобными (RISC-like), даже несмотря на то, что они имеют фиксированную длину и предположительно оптимизированы для исполнения за один такт в ядре, не нуждающемся в микрокоде. Intel предпочитает называть свою новую LIW-технология Explicitly Parallel Instruction Computing или EPIC (Вычисления с Явной Параллельностью Инструкций, где "явной" означает явно указанной при трансляции). В любом случае формат команд IA-64 не имеет ничего общего с x86. Команды x86 могут иметь длину от 8 до 108 бит, и процессор должен последовательно декодировать каждую команду после определения её границ.

- Каждый 128-битный пакет содержит шаблон (template) длиной в несколько бит, помещаемый в него компилятором, который указывает процессору, какие из команд могут выполняться параллельно. Теперь процессору не нужно будет анализировать поток команд в процессе выполнения для выявления "скрытого параллелизма". Вместо этого наличие параллелизма определяет компилятор и помещает информацию в код программы. Каждая команда (как для целочисленных вычислений, так и для вычислений с плавающей точкой) содержит три 7-битных поля регистра общего назначения (РОН). Из этого следует, что процессоры архитектуры IA-64 содержат 128 целочисленных РОН и 128 регистров для вычислений с плавающей точкой. Все они доступны программисту и являются регистрами с произвольным доступом (programmer-visible random-access registers). По сравнению с процессорами x86, у которых всего восемь целочисленных РОН и стек глубины 8 для вычислений с плавающей точкой, IA-64 намного "шире" и, соответственно, будет намного реже простаивать из-за "нехватки регистров".

- Компиляторы для IA-64 будут использовать технологию "отмеченных команд" (predication) для устранения потерь производительности из-за неправильно предсказанных переходов и необходимости пропуска участков кода после ветвлений. Когда процессор встречает "отмеченное" ветвление в процессе выполнения программы, он начинает одновременно выполнять все ветви. После того, как будет определена "истинная" ветвь, процессор сохраняет необходимые результаты и сбрасывает остальные.

- **Компиляторы для IA-64 будут также просматривать исходный код с целью поиска команд, использующих данные из памяти. Найдя такую команду, они будут добавлять пару команд - команду предварительной загрузки (speculative loading) и проверки загрузки (speculative check).** Во время выполнения программы первая из команд загружает данные в память до того, как они понадобятся программе. Вторая команда проверяет, успешно ли произошла загрузка, перед тем, как разрешить программе использовать эти данные. Предварительная загрузка позволяет уменьшить потери производительности из-за задержек при доступе к памяти, а также повысить параллелизм.

### **37. Какие факторы влияют на длину команды? Возможные пути сокращения длины команды.**

**Каждая команда должна содержать сведения, необходимые для ее выполнения.** Сведения кодируются. Для кодировки каждой группы сведений выделяется свое поле. Совокупность полей, содержащих необходимые сведения для выполнения требуемой операции, называют **форматом команды**.

**В формате команды должны быть определены:**

- **функциональное назначение операции в виде кода операции;**
- **адреса источников данных.** В общем случае должны быть указаны адреса двух операндов;
- **адрес места расположения результата;**
- **адрес следующей команды.**

Рассмотрим гипотетическую ситуацию. Допустим, что в формате команды:

- поле кода операций (КО) занимает 4 разряда, что позволяет закодировать  $2^4 = 16$  операций;
- под адреса двух операндов-источников, адрес места расположения результата, адрес следующей команды выделены поля АО 1, АО 2, АР, АСК (рис. 2.7.1) по 12 разрядов, что позволяет в каждом случае адресовать  $2^{12} = 4К$  ячеек памяти. Как видно из рис. 2.7.1, несмотря на скромные возможности команды, ее общая длина составляет достаточно большое число (52) бит.

|        |        |        |        |        |
|--------|--------|--------|--------|--------|
| КО     | АО1    | АО2    | АР     | АСК    |
| 4 бита | 12 бит | 12 бит | 12 бит | 12 бит |

**Рис. 2.7.1. Формат команды**

Для сокращения количества разрядов команды часть информации должна быть задана неявно и не должна зависеть от особенностей конкретной команды.

**Наиболее употребительными являются следующие способы сокращения длины кода команды:**

- **использование специально предусмотренных для адресации регистров: программного счетчика, указателя стека и др.** Например, при выполнении команд с последовательно возрастающими адресами программный счетчик автоматически считывает из памяти следующую команду. В этом случае в формате команды отсутствует поле адреса следующей команды;

- **использование неявных способов адресации.** Например, при использовании неявной регистровой адресации адрес следующей команды или операнда хранится в регистре, код которого содержит значительно меньшее число разрядов, чем код исполнительного адреса;

- **совмещение источника одного из операндов с приемником результата.** В этом случае в формате команды исключается поле адреса результата;

- использование команд с укороченной адресацией, т. е. части адресного пространства памяти;

- использование для некоторой группы операций одного регистра. Наиболее часто для этой цели используется аккумулятор. В операциях с аккумулятором не требуется его кодировка. Например, команда ADD выполняет операцию сложение содержимого регистра В с аккумулятором А и в него же помещает результат. Такая команда является одноадресной. Ее формат состоит из двух полей: кода операции и адреса операнда. Однако одноадресные команды требуют дополнительных команд для предварительной загрузки операндов в аккумулятор и последующего размещения результатов в памяти;

- использование нескольких аккумуляторов (Motorola 6800, National PACE, Signetic 2650). В этом случае каждая команда имеет два адреса, однако адрес источника и места назначения может быть задан другим аккумулятором. Следует отметить, что в одних процессорах все команды имеют одинаковую длину, в других — разную длину. Одинаковая длина всех команд упрощает декодирование, однако требует большего пространства, поскольку все команды должны быть такой же длины, как самая длинная. Команды могут быть короче слова, равными слову или длиннее слова. В процессорах с неймановской архитектурой команды и данные имеют одинаковую длину и поступают в процессор по шине данных. Поэтому для отличия команд, отданных в процессоре предусмотрены средства, обеспечивающие:

- засылку команд (первого байта) в регистр команд с дальнейшей их дешифрацией для активизации устройства управления;

- поступление данных (последующих байт) в аккумулятор или другие регистры для обработки в АЛУ.

### 38. В чём суть и достоинства конвейеризации транзакций?

Одним из способов повышения скорости передачи данных по шине является *конвейеризация транзакций*. Очередной элемент данных может быть отправлен устройством А до того, как устройство В завершит считывание предыдущего элемента.



Данные должны оставаться стабильными в течение времени  $t_{ст}$  (стабилизации) +  $t_{уд}$  (удержания), только после этого — смена элемента данных.

Максимальная скорость передачи —  $1/(t_{ст} + t_{уд})$

### 39. Способы ускорения транзакций.

Для сокращения времени транзакций проектировщики обычно прибегают к следующим приемам:

- арбитражу с перекрытием;
- арбитражу с удержанием шины;

- расщеплению транзакций

В шине с расщеплением транзакций линии адреса и данных обязаны быть независимыми. Каждая транзакция чтения разделяется на две части: адресную транзакцию и транзакцию данных. Считывание данных из памяти начинается с адресной транзакции: выставления ведущим на адресную шину адреса ячейки. С приходом адреса память приступает к относительно длительному процессу поиска и извлечения затребованных данных. По завершении чтения память становится ведущим устройством, запрашивает доступ к шине и направляет считанные данные по шине данных. Фактически от момента поступления запроса до момента формирования отклика шина остается незанятой и может быть востребована для выполнения других транзакций. В этом и состоит главная идея протокола расщепления транзакций. Т.о., на шине с расщеплением транзакций имеют место поток запросов и поток откликов. Часто в системах с расщеплением транзакций контроллер памяти проектируется так, чтобы обеспечить буферизацию множественных запросов.

Арбитраж с перекрытием (overlapped arbitration) заключается в том, что одновременно с выполнением текущей транзакции производится арбитраж следующей транзакции.

При арбитраже с удержанием шины (bus parking) ведущий может удерживать шину и выполнять множество транзакций, пока отсутствуют запросы от других потенциальных ведущих. В современных шинах обычно сочетаются все вышеперечисленные способы ускорения транзакций

## 40. Методы доступа к ЗУ?

При оценке быстродействия необходимо учитывать применяемый в данном типе ЗУ метод доступа к данным. Различают четыре основных метода доступа:

- Последовательный доступ. ЗУ с последовательным доступом ориентировано на хранение информации в виде последовательности блоков данных, называемых записями. Для доступа к нужному элементу (слову или байту) необходимо прочитать все предшествующие ему данные. Время доступа зависит от положения требуемой записи в последовательности записей на носителе информации и позиции элемента внутри данной записи. Примером может служить ЗУ на магнитной ленте.

- Прямой доступ. Каждая запись имеет уникальный адрес, отражающий ее физическое размещение на носителе информации. Обращение осуществляется как адресный доступ к началу записи, с последующим последовательным доступом к определенной единице информации внутри записи. В результате время доступа к определенной позиции является величиной переменной. Такой режим характерен для магнитных дисков.

- Произвольный доступ. Каждая ячейка памяти имеет уникальный физический адрес. Обращение к любой ячейке занимает одно и то же время и может вестись в произвольной очередности. Примером могут служить запоминающие устройства основной памяти.

- Ассоциативный доступ. Этот вид доступа позволяет выполнять поиск ячеек, содержащих такую информацию, в которой значение отдельных битов совпадает с состоянием одноименных битов в заданном образце. Сравнение осуществляется параллельно для всех ячеек памяти, независимо от ее емкости. По ассоциативному принципу построены некоторые блоки кэш-памяти.

## 41. Иерархия запоминающих устройств.

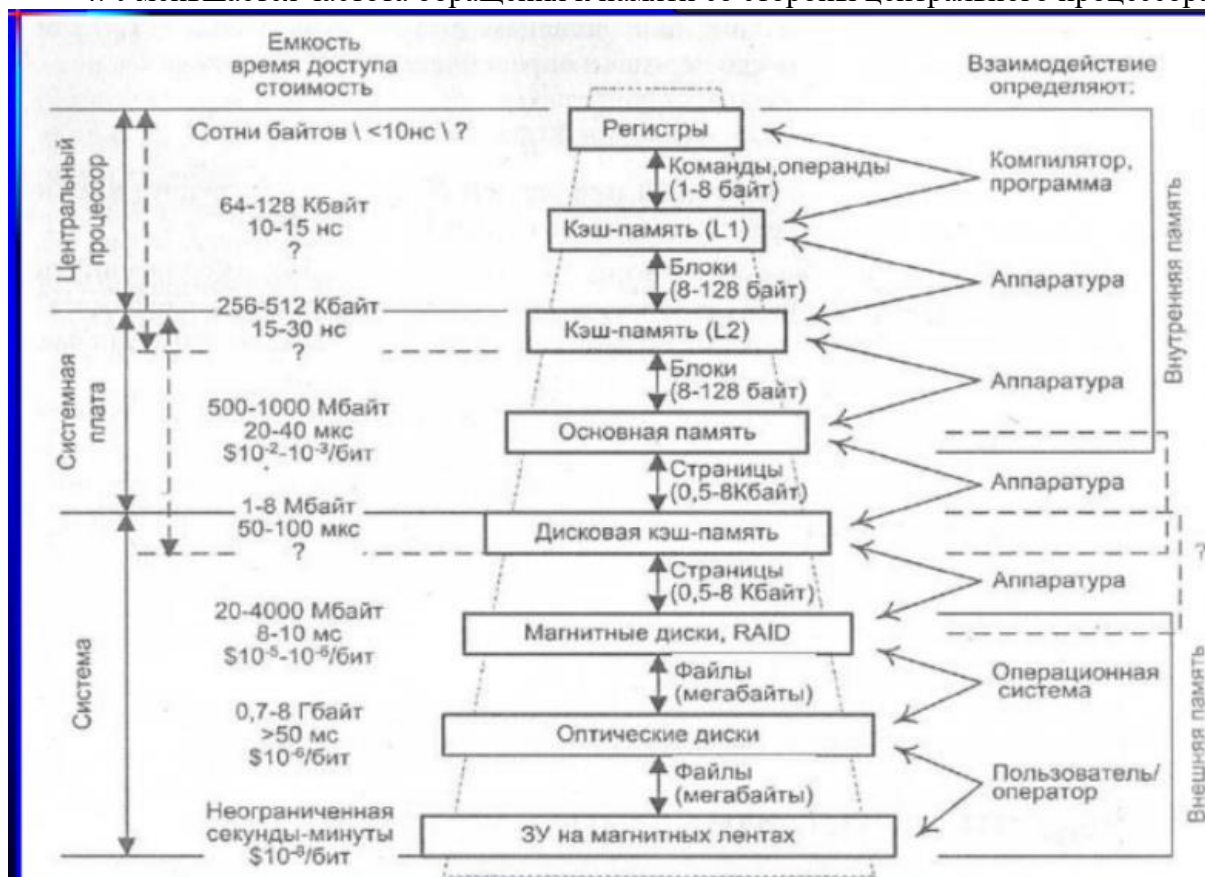
Иерархическая память состоит из ЗУ различных типов, которые, в зависимости от характеристик, относят к определенному уровню иерархии. Более высокий уровень меньше по емкости, быстрее и имеет большую стоимость в пересчете на бит, чем более низкий уровень. Уровни иерархии взаимосвязаны: все данные на одном уровне могут

быть также найдены на более низком уровне, и все данные на этом более низком уровне могут быть найдены на следующем нижележащем уровне и т. д.

Четыре верхних уровня иерархии образуют внутреннюю память ВМ, а все нижние уровни - это внешняя или вторичная память.

По мере движения вниз по иерархической структуре:

1. Уменьшается соотношение «стоимость/бит».
2. Возрастает емкость.
3. Растет время доступа.
4. Уменьшается частота обращения к памяти со стороны центрального процессора



Принцип локальности по обращению. Если рассмотреть процесс выполнения большинства программ, то можно заметить, что с очень высокой вероятностью адрес очередной команды программы либо следует непосредственно за адресом, по которому была считана текущая команда, либо расположен вблизи него. Такое расположение адресов называется пространственной локальностью программы.

Из свойства локальности вытекает, что программу разумно представить в виде последовательно обрабатываемых фрагментов - компактных групп команд и данных. Помещая такие фрагменты в более быструю память, можно существенно снизить общие задержки на обращение, поскольку команды и данные, будучи один раз переданы из медленного ЗУ в быстрое, затем могут использоваться многократно, и среднее время доступа к ним в этом случае определяется уже более быстрым ЗУ.

На каждом уровне иерархии информация разбивается на блоки, выступающие в качестве наименьшей информационной единицы, пересылаемой между двумя соседними уровнями иерархии. Размер блоков может быть фиксированным либо переменным. При фиксированном размере блока емкость памяти обычно кратна его размеру. Размер блоков на каждом уровне иерархии чаще всего различен и увеличивается от верхних уровней к нижним.

При доступе к командам и данным, например, для их считывания, сначала производится поиск в памяти верхнего уровня. Факт обнаружения нужной информации называют попаданием (hit), в противном случае говорят о промахе (miss). При промахе

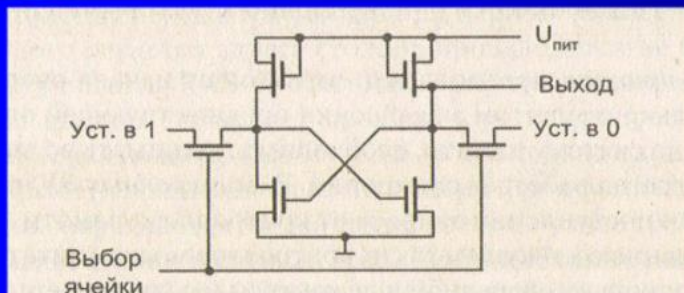
производится поиск в ЗУ следующего более низкого уровня, где также возможны попадание или промах

## 42. Статические и динамические ОЗУ?

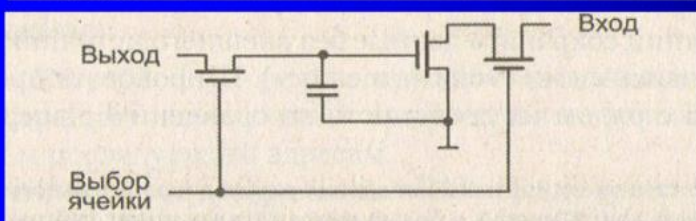
В **статических ОЗУ** запоминающий элемент может хранить записанную информацию неограниченно долго (при наличии питающего напряжения).

Запоминающий элемент **динамического ОЗУ** способен хранить информацию только в течение достаточно короткого промежутка времени, после которого информацию нужно восстанавливать заново, иначе она будет потеряна. Динамические ЗУ, как и статические, энергозависимы.

Роль запоминающего элемента в статическом ОЗУ выполняет триггер (схема с двумя устойчивыми состояниями, обычно состоящую из четырех или шести транзисторов).



SRAM, на 6 транзисторах – быстрее, менее чувствительны к внешним излучениям



DRAM

Простота схемы DRAM позволяет достичь высокой плотности размещения, в итоге, *снизить стоимость*. *Главный недостаток - что накапливаемый на конденсаторе заряд со временем теряется. Среднее время утечки заряда DRAM составляет сотни(десятки) миллисекунд, поэтому, заряд следует регенерировать.*

Слайд 2

Рассмотрим достоинства и недостатки SRAM и DRAM:

1. Память типа DRAM, в силу своей технологии, имеет гораздо большую плотность размещения данных, чем SRAM.
2. DRAM гораздо дешевле SRAM,
3. но последняя производительнее и надежнее, поскольку всегда готова к считыванию.

### 43. Блочная схема расслоения памяти.

Емкость современных ОЗУ слишком велика для одной ИМС. Также объединение нескольких ИМС ЗУ необходимо если разрядность ИМС меньше разрядности ШД. Если объединить несколько ИМС по адресным входам до нужной разрядности - это модуль памяти (может состоять и из одной ИМС).

Один или несколько модулей - банк памяти.

Для получения ЗУ нужной ёмкости – объединение нескольких банков памяти. Адрес ячейки  $A$  памяти из  $B$  банков  $\rightarrow (b, w)$ , где  $b$  – номер банка,  $w$  – адрес ячейки в банке.

Три схемы распределения адресов  $A$  на  $(b, w)$ :

*Блочная (номер банка  $b$  – старшие разряды адреса)*

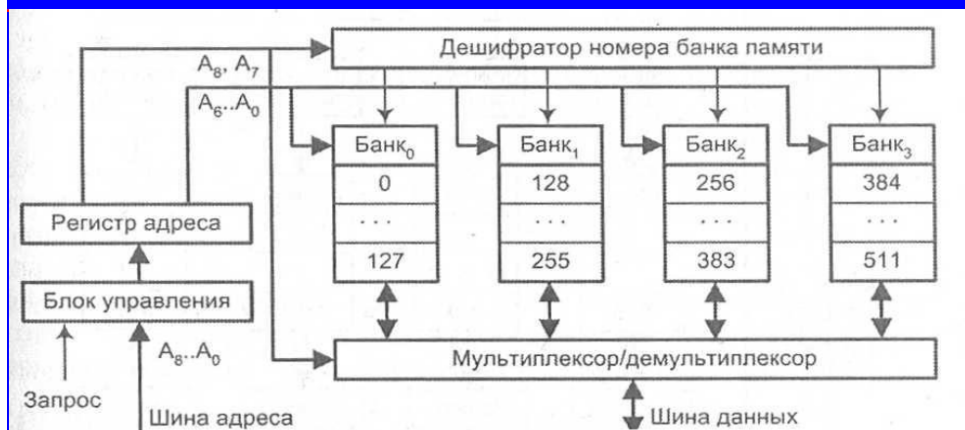
*Циклическая ( $b = A \bmod B$ ;  $w = A \div B$ )*

*Блочно-циклическая (комбинация двух предыдущих схем)*

Пример – память ёмкостью 512 слов – четыре банка по 128 слов в каждом (типовая структура по блочному принципу).

По данному:

### Блочная организация памяти



Адресное пространство памяти разбито на группы последовательных адресов. Каждая такая группа обеспечивается отдельным банком памяти. Для обращения используется 9-разрядный адрес, семь младших разрядов которого ( $A_6 - A_0$ ) поступают параллельно на все банки памяти и выбирают в каждом из них одну ячейку.

Два старших разряда адреса ( $A_8, A_7$ ) содержат номер банка. Выбор банка обеспечивается либо с помощью дешифратора номера банка памяти, либо путем мультиплексирования информации рис. иллюстрирует оба варианта). В функциональном отношении такая ОП может рассматриваться как единое ЗУ, емкость которого равна суммарной емкости отдельных банков, а быстродействие – быстродействию отдельного банка.

#### 44. Циклическая схема расслоения памяти.

Начало 43 вопроса;

В каждом такте на шине адреса может присутствовать адрес только одной ячейки, поэтому параллельное обращение к нескольким банкам невозможно, но оно может быть организовано со сдвигом на один такт. Адрес ячейки запоминается в индивидуальном регистре адреса, и дальнейшие операции по доступу к ячейке в каждом банке протекают независимо. При большом количестве банков среднее время доступа к памяти сокращается почти в  $B$  раз ( $B$  – количество банков), но при условии, что ячейки относятся к разным банкам. Если же запросы к одному тому же банку следуют друг за другом, каждый следующий запрос должен ожидать завершения обслуживания предыдущего. Такая ситуация называется конфликтом по доступу. При частых конфликтах данный метод становится неэффективным.

#### 45. Блочно-циклическая схема расслоения памяти.

Начало 43 вопроса;

**Блочно-циклическая схема расслоения** – каждый банк – из нескольких модулей, адресуемых по круговой схеме. Адреса между банками – по блочной схеме. В итоге – адрес ячейки разбивается на три части: *номер банка, с модуле, модуль в банке.*

| Банк <sub>0</sub>   |                     | Банк <sub>1</sub>   |                     | Банк <sub>2</sub>   |                     | Банк <sub>3</sub>   |                     |
|---------------------|---------------------|---------------------|---------------------|---------------------|---------------------|---------------------|---------------------|
| Модуль <sub>0</sub> | Модуль <sub>1</sub> | Модуль <sub>2</sub> | Модуль <sub>3</sub> | Модуль <sub>4</sub> | Модуль <sub>5</sub> | Модуль <sub>6</sub> | Модуль <sub>7</sub> |
| 0                   | 1                   | 128                 | 129                 | 256                 | 257                 | 384                 | 385                 |
| 2                   | 3                   | 130                 | 131                 | 258                 | 259                 | 386                 | 387                 |
| ...                 | ...                 | ...                 | ...                 | ...                 | ...                 | ...                 | ...                 |
| 126                 | 127                 | 254                 | 255                 | 382                 | 383                 | 510                 | 511                 |

Расслоение хорошо работает для обычной задачи – благодаря свойству локальности. В многопроцессорных системах с общей памятью – несколько контроллеров для автономной работы – по отдельным банкам. Чем больше банков – тем меньше вероятность конфликта. В суперкомп-ре NEC SX/3 – 128 банков основной памяти.<sup>2</sup>

#### 46. Последовательный и конвейерный режим доступа к микросхеме ЗУ.

Немного воды:

Интегральные микросхемы (ИМС) памяти организованы в виде матрицы ячеек, каждая из которых в зависимости от разрядности ИМС состоит из одного или более запоминающих элементов (ЗЭ).

Каждый ЗЭ способен хранить 1 бит информации. При матричной организации ИМС памяти реализуется координатный принцип адресации ячеек.

Адрес ячейки разделяется логикой выбора на две составляющие адрес строки и адрес столбца. Адреса запоминаются в соответствующих регистрах. Дешифраторы регистров выбирают вертикальные и горизонтальные линии, на пересечении которых находятся по одному ЗЭ.

##### Последовательный режим

При использовании последовательного режима (Flow through Mode) адрес и управляющие сигналы подаются на микросхему до поступления синхроимпульса. В

момент прихода синхроимпульса вся входная информация запоминается во внутренних регистрах — по его переднему фронту, и начинается цикл чтения. Через некоторое время, но в пределах того же цикла данные появляются на внешней шине, причем момент этот определяется только моментом прихода синхронизирующего импульса и скоростью внутренних цепей микросхемы.

#### Конвейерный режим

Конвейерный режим (pipelined mode) - это такой метод доступа к данным, при котором можно продолжать операцию чтения по предыдущему адресу в процессе запроса по следующему.

При чтении из памяти время, требуемое для извлечения данных из ячейки, можно условно разбить на два интервала. Первый из них — непосредственно доступ к массиву запоминающих элементов и извлечение данных из ячейки. Второй — передача данных на выход (при этом происходит детектирование состояния ячейки, сигнала и другие операции, необходимые для считывания информации).

В отличие от последовательного режима, где следующий цикл чтения начинается только по окончании предыдущего, в конвейерном режиме процесс разбивается на два этапа. Пока данные из предыдущего цикла чтения передаются на внешнюю шину, происходит запрос на следующую операцию чтения. Таким образом, два цикла чтения перекрываются во времени. Из-за усложнения схемы передачи данных на внешнюю шину время считывания увеличивается на один такт, и данные поступают на выход только в следующем такте, но такое запаздывание наблюдается лишь при первом чтении в последовательности операций считывания из памяти. Все последующие данные поступают на выход друг за другом, хотя и с запаздыванием на один такт относительно запроса на чтение. Так как циклы чтения перекрываются, микросхемы с конвейерным режимом могут использоваться при частотах шины, вдвое превышающих допустимую для ИМС с последовательным режимом чтения.

## **47. Регистровый и страничный режим доступа к микросхеме 3У.**

#### Регистровый режим

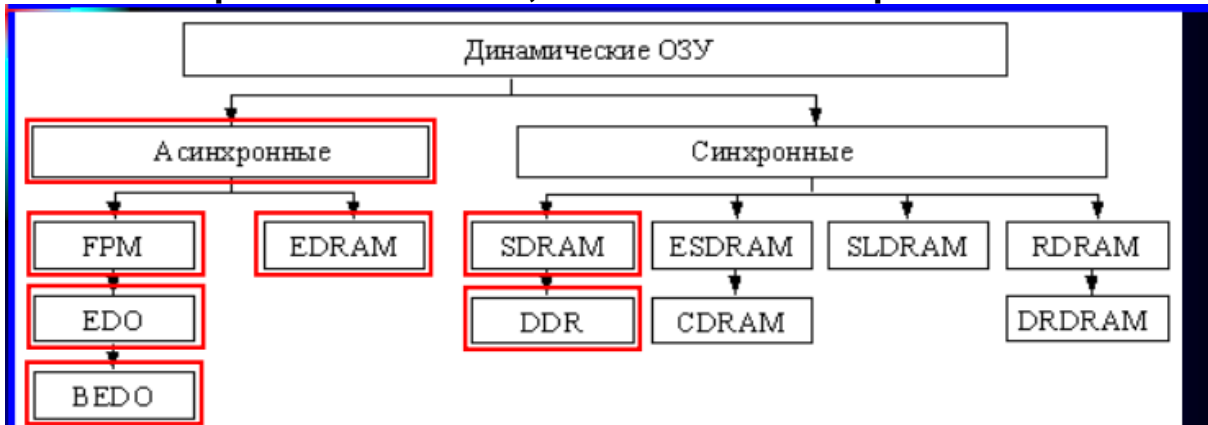
Регистровый режим (Register to Latch) используется относительно редко и отличается наличием регистра на выходе микросхемы. Адрес и управляющие сигналы выдаются на шину до поступления синхронизирующего импульса. С приходом положительного фронта синхроимпульса адрес записывается во внутренний регистр микросхемы, и начинается цикл чтения. Считанные данные заносятся в промежуточный выходной регистр и хранятся там до появления отрицательного фронта (спада) синхроимпульса, а с его поступлением передаются на шину. Метод однозначно определяет момент появления данных на выходе ИМС, причем изменяя ширину импульса синхронизации можно менять время появления данных на шине. Данное свойство часто оказывается весьма полезным при проектировании специализированных ВМ. По быстродействию микросхемы с регистровым режимом идентичны ИМС с последовательным режимом.

#### Страничный режим

В основе идеи лежит тот факт, что при доступе к ячейкам со смежными адресами (согласно принципу локальности, такая ситуация наиболее вероятна), причем к таким, где все 3Э расположены в одной строке матрицы, доступ ко второй и последующим ячейкам можно производить существенно быстрее. Действительно, если адрес строки при очередном обращении остался прежним, то все временные затраты, связанные с повторным занесением адреса строки в регистр ИМС, дешифровкой, зарядом паразитной емкости горизонтальной линии и т. п., можно исключить. Для доступа к очередной ячейке достаточно подавать на ИМС лишь адрес нового столбца, сопровождая его сигналом CAS. Отметим, что обращение к первой ячейке в последовательности производится

стандартным образом — поочередным заданием адреса строки и адреса столбца, то есть здесь время доступа уменьшить практически невозможно. Рассмотренный режим называется режимом страничного доступа или просто страничным режимом (Page Mode). Под страницей понимается строка матрицы ЗЭ. Микросхемы, где реализуется страничный режим и его модификации, принято характеризовать формулой х-у-у-у. Первое число х представляет количество тактов системной шины, необходимое для доступа к первой ячейке последовательности, а у — к каждой из последующих ячеек. Так, выражение 7-3-3-3 означает, что для обработки первого слова необходимо 7 тактовых периодов системной шины (в течение шести из которых шина простаивает в ожидании), а для обработки последующих слов — по три периода, из которых два системная шина также простаивает.

#### 48. Микросхемы SDRAM, отличия от асинхронных DRAM.



**DDR SDRAM – Double Data Rate Synchronous DRAM.**  
 Главное отличие – данные в **пакетном режиме** выдаются по обоим фронтам импульса синхронизации, за счёт чего пропускная способность возрастает вдвое.

Микросхемы DDR SDRAM (Double Data Rate SDRAM - SDRAM с удвоенной скоростью передачи данных) - дальнейшее развитие технологии SDRAM - выдает данные в пакетном режиме по обоим фронтам импульса синхронизации, увеличивая пропускную способность вдвое.

Сущ. несколько спецификаций DDR SDRAM в зависимости от тактовой частоты шины – DDR266, DDR333, DDR400, DDR533. Пиковая пропускная способность DDR333 – 2,7 Гбайт/с, DDR400 – 3,2 Гбайт в сек.

Микросхемы SDRAM. Аббревиатура SDRAM (Synchronous DRAM - синхронная DRAM) используется для обозначения микросхем «обычных» синхронных динамических ОЗУ. Кардинальные отличия SDRAM от рассмотренных выше асинхронных динамических ОЗУ можно свести к четырем положениям:

- синхронный метод передачи данных на шину;
- конвейерный механизм пересылки пакета;
- применение нескольких (двух или четырех) внутренних банков памяти;

- передача части функций контроллера памяти логике самой микросхемы.

Синхронность памяти позволяет контроллеру памяти «знать» моменты готовности данных, за счет чего снижаются издержки циклов ожидания и поиска данных. Так как данные появляются на выходе ИМС одновременно с тактовыми импульсами, упрощается взаимодействие памяти с другими устройствами ВМ.

Значительный эффект дает разбиение всей совокупности ячеек на независимые внутренние массивы (банки). Это позволяет совмещать доступ к ячейке одного банка с подготовкой к следующей операции в остальных банках (перезарядкой управляющих цепей и восстановлением информации). Возможность держать открытыми одновременно несколько строк памяти (из разных банков) также способствует повышению быстродействия памяти. При поочередном доступе к банкам частота обращения к каждому из них в отдельности уменьшается пропорционально числу банков и SDRAM может работать на более высоких частотах. Благодаря встроенному счетчику адресов SDRAM, позволяет производить чтение и запись в пакетном режиме, причем в SDRAM длина пакета варьируется и в пакетном режиме есть возможность чтения целой строки памяти. ИМС может быть охарактеризована формулой 5-1-1-1.

## **49. Энергонезависимые ОЗУ.**

Под понятие энергонезависимое ОЗУ (NVRAM - Non-Volatile RAM) подпадает несколько типов памяти. От перепрограммируемых постоянных ЗУ их отличает отсутствие этапа стирания, предваряющего запись новой информации, поэтому вместо термина «программирование» для них употребляют стандартный термин «запись».

Микросхемы BBSRAM. К рассматриваемой группе относятся обычные статические ОЗУ со встроенным литиевым аккумулятором и усиленной защитой от искажения информации в момент включения и отключения внешнего питания. Для их обозначения применяют аббревиатуру BBSRAM (Battery-Back SRAM).

Микросхемы NVRAM. Другой подход реализован в микросхеме, разработанной компанией Simtec. Особенность ее в том, что в одном корпусе объединены статическое ОЗУ и перепрограммируемая постоянная память типа EEPROM. При включении питания данные копируются из EEPROM в SRAM, а при выключении — автоматически перезаписываются из SRAM в EEPROM. Благодаря такому приему данный вид памяти можно считать энергонезависимым.

Микросхемы FRAM. FRAM (Ferroelectric RAM - ферроэлектрическая память) разработана компанией Ramtron и представляет собой еще один вариант энергонезависимой памяти. По быстродействию данное ЗУ несколько уступает динамическим ОЗУ и пока рассматривается лишь как альтернатива флэш-памяти. Причисление FRAM к оперативным ЗУ обусловлено отсутствием перед записью явно выраженного цикла стирания информации.

## **50. Специализированные ОЗУ.**

В ряде практических задач более выгодным оказывается использование специализированных архитектур ОЗУ, где стандартные функции (запись, хранение, считывание) сочетаются с некоторыми дополнительными возможностями или учитывают особенности применения памяти. Такие виды ОЗУ называют специализированными и к ним причисляют:

- память для видеоадаптеров;
- память с множественным доступом (многопортовые ОЗУ);
- память типа очереди (ОЗУ типа FIFO).

Использование памяти в видеоадаптерах имеет свою специфику и для реализации дополнительных требований прибегают к несколько иным типам микросхем. Так, при создании динамичных изображений часто достаточно просто изменить расположение уже хранящейся в видеопамяти информации. Вместо того чтобы многократно пересылать по

шине одни и те же данные, лишь несколько изменив их расположение, выгоднее заставить микросхему памяти переместить уже хранящиеся в ней данные из одной области ядра в другую. На ИМС памяти можно также возложить операции по изменению цвета точек изображения (SGRAM, VRAM, WRAM, MDRAM, 3D-RAM)

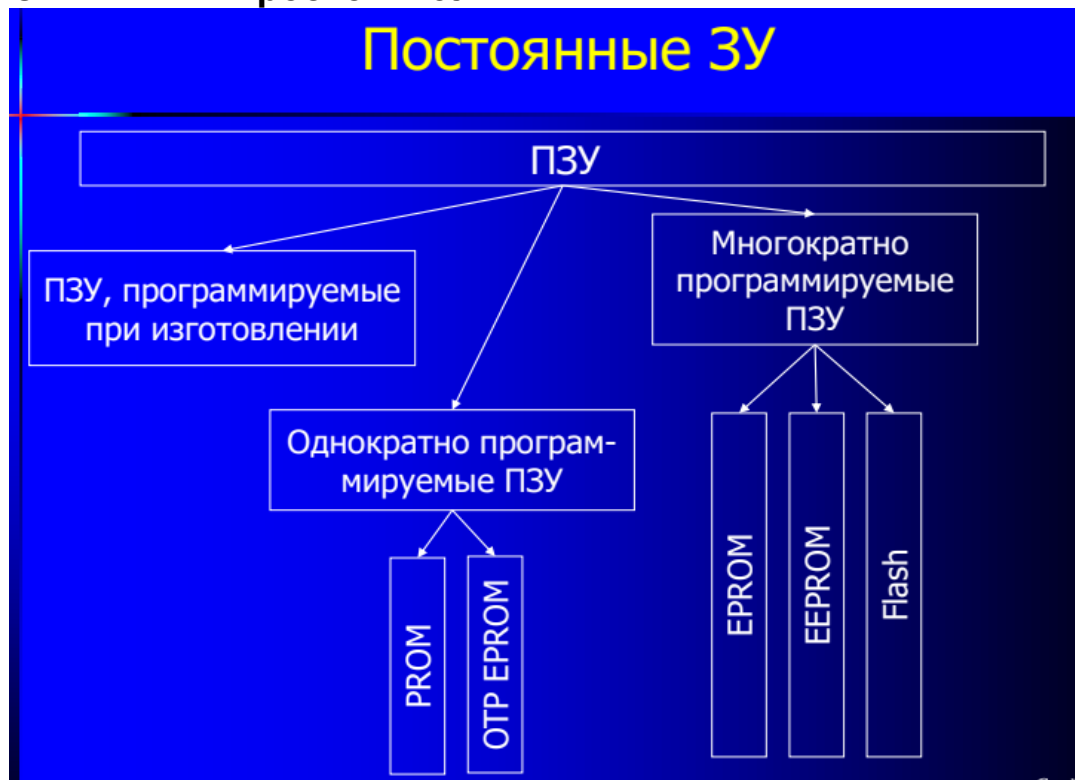
В отличие от стандартного в n-портовом ОЗУ имеется п независимых наборов шин адреса, данных и управления, гарантирующих одновременный и независимый доступ к ОЗУ п устройствам. Данное свойство позволяет существенно упростить создание многопроцессорных и многомашинных вычислительных систем, где многопортовое ОЗУ выступает в роли общей или совместно используемой памяти. В рамках одной ВМ подобное ОЗУ может обеспечивать обмен информацией между ЦП и УВВ (например, контроллером магнитного диска) намного эффективней, чем прямой доступ к памяти.

Во многих случаях ОЗУ применяется для буферизации потока данных, когда данные считываются из памяти в той же последовательности, в которой они туда заносились, но поступление и считывание происходят с различной скоростью. Часто для этой цели применяют обычное ОЗУ, однако здесь одновременная запись и считывание информации невозможны. Более эффективным видом ОЗУ, где оба действия могут вестись одновременно, служит память типа FIFO. Микросхема представляет собой двухпортовое ОЗУ, где один порт предназначен для занесения информации, а второй - для считывания. Для FIFO-памяти характерны все технологические приемы, свойственные двухпортовой памяти, в частности способы арбитража при одномоментном обращении к одной и той же ячейке. В то же время есть и существенные отличия.

Первое состоит в том, что у микросхемы нет входов для указания адреса ячейки, занесение и считывание данных производится в порядке их поступления через одну входную точку и одну выходную.

Второе отличие связано с необходимостью слежения за состоянием очереди. Для этого в микросхеме имеются регистры-указатели адресов начала и конца очереди, а также специальные флаги, которые указывают на две ситуации: отсутствие данных (в этом случае блокируется считывание из микросхемы) и полное заполнение памяти (блокируется запись).

## 51. Типы микросхем ПЗУ.



ПЗУ – **Read Only Memory (ROM)**. Программируемые при изготовлении – MROM (Mask ROM).

Занесение информации в ПЗУ – часть технологического процесса изготовления микросхемы. Различные технологии, но чаще всего – отсутствие либо наличие перемычки (транзистора) на пересечении адресной и разрядной линии. Наиболее дешёвый вид ПЗУ – высокая плотность упаковки ЗЭ на кристалле и высокая скорость считывания. Пример применения – ПЗУ шрифтов в лазерных принтерах.

Однократно программируемые ПЗУ – полуфабрикат, занесение информации – на этапе изготовления мелкосерийного устройства, использующего ПЗУ.

PROM – **Programmable ROM**. Первые микросхемы – на базе плавких предохранителей. Занесение информации – пережигание отдельных перемычек (пробоя отдельных диодов). Основной недостаток – необходимость спец. программатора, большой процент брака, необходимость термической постобработки схемы (- иначе низкая надёжность хранения).

ОТР EPROM – **One Time Programmable EPROM** – однократно программируемая EPROM – кристалл EPROM в пластиковом корпусе (без кварцевого окна).

## Многократно программируемые ПЗУ

В многократно программируемых два этапа – стирание всего + запись новой информации.

EPROM – **Erasable Programmable ROM** – запись электрическими сигналами, но стирание – воздействием ультрафиолетового облучения через кварцевое окно корпуса микросхемы. Для исключения случайного стирания – заклеивание окна непрозрачной плёнкой. Стирание – многократное (20 мин). Иногда называют UV-EPROM - **Ultra Violet EPROM**. Время программирования – несколько сотен миллисекунд. Время считывания как у ROM и DRAM. Дороже, чем PROM, но выгоднее в итоге.

**EEPROM – Electrical Erasable Programmable ROM** – стирание и запись по-байтово, причём стирание – как обязательный этап записи. Запись – намного дольше, чем чтение – несколько сотен микросекунд на байт. Не требует программатора – запись реализуется внутренней логикой микросхемы.

Два варианта – с последовательным и параллельным доступом. 90% всех микросхем – **SEEPROM – Serial EEPROM** – адреса, данные и управление – по одному входу. Преимущества – малые габариты, минимальное число линий ввода/вывода. Недостатки – большое время доступа, цена – EEPROM дороже EPROM.

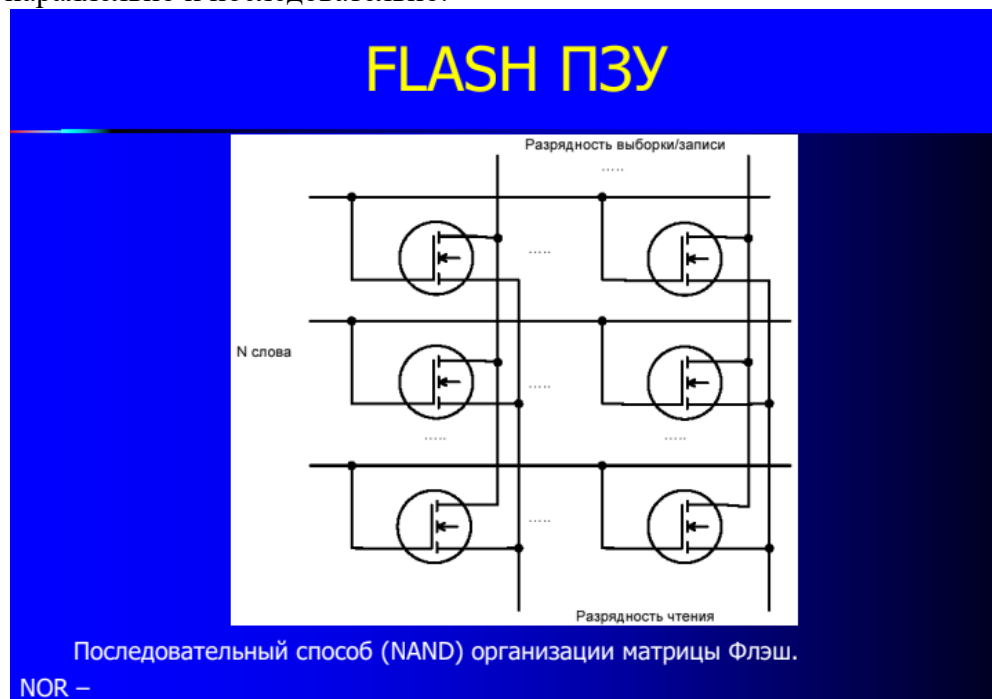
## 52. «Flash» ПЗУ.

FLASH - Флэш-память появилась в конце 1980-х годов (Intel) – перепрограммируемая ПЗУ с электрическим стиранием. Стирание осуществляется сразу целой областью ячеек : блока или всей микросхемы => более быстрая запись информации (программирование ЗУ).

Для упрощения записи – спец. блоки ИМС, делающие запись “прозрачной” для ПО и внешних устройств.

Флэш-память строится на однотранзисторных элементах памяти (с “плавающим” затвором), => плотность хранения информации чуть выше, чем DRAM.

Различные технологии построения базовых элементов флэш - количество слоев, методы стирания и записи данных, структура. Наиболее широко известны NOR и NAND типы флэш-памяти, запоминающие транзисторы в которых подключены к разрядным шинам - параллельно и последовательно.



Минусы – относительно невысокая скорость передачи данных.

Элементы памяти флэш-ЗУ организованы в матрицы, разрядность данных для микросхем - 1-2 байта.

Операция чтения из флэш - как в обычных ЗУ с произвольным доступом (ОЗУ). Но запись – с некоторыми особенностями, аналогичными - ПЗУ. Перед записью данных в ячейки они должны быть очищены (стерты). Стирание - перевод элементов памяти в состояние единицы и возможно только сразу для целого блока ячеек (в первых микросхемах предусматривалось стирание только для всей матрицы сразу- Bulk Erase). Выборочное - невозможно. В процессе записи информации соответствующие элементы памяти переключаются в нулевое состояние. Также, как и в ПЗУ, без стирания можно дозаписать нули в уже запрограммированные ячейки, однако необходимость в такой операции относительно редка.

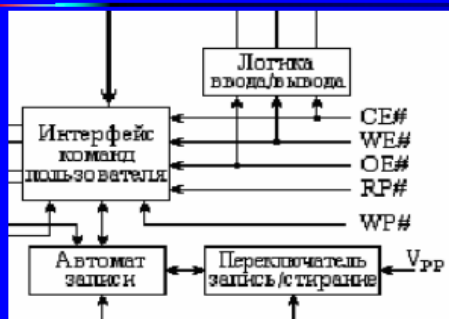
Фактически при операции записи производится два действия: стирание и запись, но управление этими операциями производится внутренним автоматом и “прозрачно” для процессора.

Разбиение адресного пространства микросхемы флэш –памяти на блоки обычно двух видов: симметричное и асимметричное.

Количество блоков зависит от ёмкости микросхемы. Например, в микросхеме 28F128J3 (Intel Strata Flash) емкостью 128 Мбит (16 Мбайт) имеется 128 блоков по 128 Кб.

Пример - 28F640C3 (Intel Advanced+ Boot Block) ёмкостью 64 Мбит: загрузочный (Boot) блок размером 64 Кбайт, разбит на 8 блоков (parameter blocks) по 8 Кбайт, и 127 основных (main) блоков по 64 Кбайт. Загрузочный блок- либо в начале, либо в конце адресного пространства ИМС.

## FLASH ПЗУ



WP# (Write Protect) используется для исключения возможности случайной записи по командам программы. RP# (Reset/Deep Power Down) - закрывает все блоки от записи. На вход  $V_{PP}$  подается напряжение, необходимое для ускорения операций стирания и записи данных.

Для улучшения характеристик в флэш-памяти предусматриваются:

1)

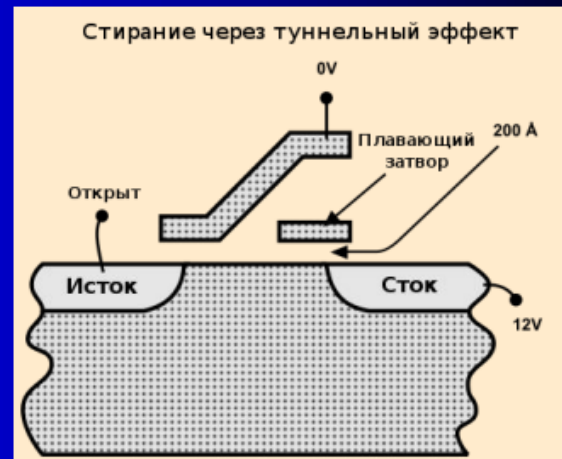
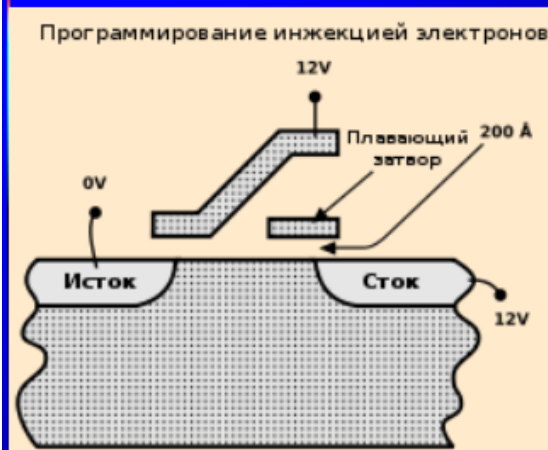
Для улучшения характеристик в флэш-памяти:

- 4) использование режимов понижения мощности когда к ЗУ нет обращения (важно для мобильных устройств);
- 5) приспособленность к работе с различными питающими напряжениями;
- 6)

Применение – BIOS, «твердотельные диски» (solid-state disks), карты памяти мобильных устройств.

Время записи байта – 10 мкс, время доступа при чтении – 35-200нс, полное стирание 1-2 с, высокая плотность размещения на кристалле – на 30% больше чем у DRAM.

# FLASH ПЗУ



## 53. Производительность иерархии памяти.

При доступе к командам и данным, например, для их считывания, сначала производится поиск в памяти верхнего уровня. Факт обнаружения нужной информации называют попаданием (hit), в противном случае говорят о промахе (miss). При промахе производится поиск в ЗУ следующего более низкого уровня, где также возможны попадание или промах.

## Производительность иерархии памяти

Основная формула:

- $T_{avg} = P_{hit} * T_{hit} + P_{miss} * T_{miss}$

$T_{hit}$  = время выполнения запроса данных из памяти в случае, если данные присутствуют на текущем уровне иерархии

$T_{miss}$  = время выполнения запроса данных в случае если данные на текущем уровне не присутствуют и необходимо обращение к нижестоящему уровню

$P_{hit}, P_{miss}$  = вероятности попадания (hit) или промаха (miss) для текущего уровня

–  $P_{hit}$  всегда 100% для самого нижнего уровня иерархии